Apprentissage Statistique Automatique II

Lucas Reding, Thibault Defourneau

Lecture note

Contents

1 Introduction **2**

2 Tensorflow **3**
2.1 Basics . . . 3
2.2 Loss wrapper . . . 4
2.3 Regularizer wrapper . . . 4
2.4 Metric wrapper . . . 5
2.5 Custom layer with Keras . . . 5

3 Optimizers **7**
3.1 Momentum methods . . . 8
3.2 Adaptive optimizers . . . 9
3.2.1 AdaGrad . . . 9
3.2.2 RMSProp . . . 9
3.2.3 Adam . . . 9

4 Regularization **11**
4.1 Regularization term . . . 11
4.2 Dropout . . . 13
4.3 Data augmentation and normalization . . . 16

5 Convolutional Neural Networks (CNNs) **18**
5.1 Convolution product . . . 18
5.2 Convolution layer . . . 20
5.2.1 Matrix representation of a convolution . . . 21
5.2.2 Translation equivariance . . . 22
5.2.3 Pointwise activation functions . . . 23
5.2.4 Equivariance and invariance . . . 23
5.2.5 Effect of the stride . . . 24
5.3 Pooling and Unpooling layers . . . 24
5.3.1 Max and average pooling . . . 24
5.3.2 Unpooling . . . 25
5.4 Definition of a CNN . . . 26
5.5 Receptive field . . . 26
5.5.1 Receptive field and effective jump . . . 27
5.5.2 The case of unit stride . . . 28
5.6 Brief historical evolution of convolutional networks . . . 28
5.6.1 LeNet-5 . . . 29
5.6.2 AlexNet . . . 30
5.6.3 VGGNets . . . 31
5.6.4 ResNet . . . 32

6 Transformer-based models **34**
6.1 Tokenization and dense embeddings . . . 34

6.2 Scaled dot-product self-attention . . . 35
6.3 Attention masks and causal language modelling . . . 36
6.4 Multi-head attention . . . 36
6.5 Why positional information is necessary . . . 37
6.5.1 Sinusoidal positional encoding . . . 37
6.5.2 Rotary positional embeddings . . . 38
6.6 A Transformer block . . . 38
6.7 Encoder-only, decoder-only, and encoder–decoder models . . . 39
6.8 Training objective and token generation . . . 40
6.9 Computational considerations . . . 40
6.10 Exercises . . . 40

7 Large Language Models **42**
7.1 Autoregressive language modelling and tokenization . . . 42
7.2 Pre-training objective . . . 44
7.3 Scaling laws, data, and compute . . . 46
7.4 Post-training: instruction tuning and preference alignment . . . 47
7.5 Decoding, evaluation, and reliability . . . 49
7.6 Capabilities and limitations . . . 52
7.7 Exercises . . . 53

8 Multimodal extensions of LLMs **55**
8.1 From images to visual tokens . . . 55
8.2 Contrastive vision-language models: CLIP . . . 56
8.3 Generative vision-language models: BLIP-2 . . . 58
8.4 Exercises . . . 60

References **62**

1 Introduction

Deep learning is a family of learning methods that model data with complex architectures combining many non-linear transformations. The elementary building blocks of deep learning are artificial neural networks, which are composed to form deep neural networks.

These techniques have enabled significant progress in sound and image processing, including facial recognition, speech recognition, computer vision, natural language processing and text classification (for example spam detection). Potential applications are very numerous. A spectacular early example is the AlphaGo program, which learned to play the game of Go with deep reinforcement learning and defeated Lee Sedol, one of the strongest players in the world, in 2016. More recently, large language models have brought deep learning into everyday tools.

There exist several families of neural network architectures:

- multilayer perceptrons, which are the oldest and simplest ones;
- recurrent neural networks, historically used for sequential data such as text or time series;
- convolutional neural networks (CNNs), particularly well suited to image processing;
- Transformers, the architecture behind large language models (LLMs), which are now widely used in many applications.

All of them are based on deep cascades of layers. They require well-designed stochastic optimization algorithms, careful initialization and a careful choice of architecture. They lead to very impressive results, although their theoretical understanding remains partial.

These notes cover the practical tools used to train neural networks (Sections 2 and 3), regularization (Section 4), convolutional neural networks (Section 5), Transformers (Section 6), large language models (Section 7) and their multimodal extensions (Section 8).

2 Tensorflow

2.1 Basics

TensorFlow is a framework developed by Google Brain and designed for machine learning. It is the core tool behind well-known services such as Google Cloud Speech, Google Photos and Google Search. The following lines of code present the main components of this framework:

```
import tensorflow as tf

# High-level deep learning API
tf.keras

# Low-level deep learning API
tf.nn

# Non-exhaustive list of NumPy-like modules
tf.math
tf.linalg
tf.signal
tf.random
tf.bitwise

# Data processing
tf.data
tf.audio
tf.image
tf.io
tf.queue

# Optimization and deployment
tf.distribute
tf.saved_model
tf.autograph
tf.graph_util
tf.lite
tf.quantization
tf.tpu

# Special data structures
tf.lookup
tf.nest
tf.ragged
tf.sets
tf.sparse
tf.strings
```

Remark 1. *At the lowest level, each TensorFlow operation is implemented in C++. Moreover, many operations have several implementations, called kernels. Each kernel is dedicated to a specific type of processor, such as a CPU, a GPU or a TPU (Tensor Processing Unit).*

Remark 2. *The code in these notes follows the Keras 3 API, which is the default Keras version of recent TensorFlow releases. Minor adaptations may be needed with older versions.*

Using TensorFlow is a similar experience to using NumPy, although the names of some basic operations differ. Let us compare a few equivalent operations in TensorFlow and NumPy:

```
import numpy as np
import tensorflow as tf

# Matrix definition
tensor = tf.constant([[1, 2, 3], [4, 5, 6]]) # in TensorFlow
arr = np.array([[1, 2, 3], [4, 5, 6]]) # in NumPy
arr_from_tensor = tensor.numpy() # The NumPy version of a tensor can be retrieved

# Matrix multiplication
tensor @ tf.transpose(tensor) # in TensorFlow
arr @ arr.T # in NumPy

# Mean computation
tf.reduce_mean(tensor) # in TensorFlow
```

```
np.mean(arr) # in NumPy
```

Remark 3. *In the above examples, the variable* `tensor` *is a* `tf.Tensor` *object, which is immutable. However, some objects must be modified in place, for instance the trainable weights of a model, which are updated at every training step. This is why the class* `tf.Variable` *exists.*

```
import numpy as np
import tensorflow as tf

var = tf.Variable([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]])
arr = np.array([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]])

var.assign(2 * var) # In TensorFlow, the method assign updates var in place
arr = 2 * arr # In NumPy, we simply use the assignment '='
```

Exercise 1. *Implement the Huber loss in TensorFlow using only basic operations. The Huber loss is defined as follows:*

$$\text{Huber}(x) = \begin{cases} \frac{1}{2}\,x^2 & ,\, if\, |x| \leqslant 1 \\ |x| - \frac{1}{2} & ,\, if\, |x| > 1 \end{cases}$$

2.2 Loss wrapper

A custom loss can be defined by subclassing `tf.keras.losses.Loss`, with two methods to customize:

- the method `call` receives the targets and the predictions and computes the losses during the training step;
- the method `get_config` returns a dictionary which associates each hyperparameter name with its value.

Let us define the Huber loss with this approach:

```
import tensorflow as tf

class HuberLoss(tf.keras.losses.Loss):
    def __init__(self, threshold=1.0, **kwargs):
        self.threshold = threshold
        super().__init__(**kwargs)

    def call(self, y_true, y_pred):
        computed_loss = ...  # To be completed with the logic of Exercise 1
        return computed_loss

    def get_config(self):
        base_config = super().get_config()
        return {**base_config, "threshold": self.threshold}
```

This way of defining a custom loss is very practical, because all the necessary abstractions are already handled by the base class `tf.keras.losses.Loss`. In particular, the hyperparameters of the custom loss are saved together with the trained model:

```
import tensorflow as tf

# Let `model` be any Keras model. When compiling it with the custom loss, it is enough to write:
model.compile(loss=HuberLoss(threshold=2.0), optimizer="nadam")
model.save("my_model_with_the_custom_huber_loss.keras")

# When loading the trained model, the custom loss must be provided as well:
model = tf.keras.models.load_model(
    "my_model_with_the_custom_huber_loss.keras",
    custom_objects={"HuberLoss": HuberLoss},
)
```

2.3 Regularizer wrapper

Similarly, a custom regularizer can be defined by subclassing `tf.keras.regularizers.Regularizer`. As an example, let us define the ℓ_1 regularization (see Section 4.1):

```python
import tensorflow as tf

class L1Regularizer(tf.keras.regularizers.Regularizer):
    def __init__(self, factor):
        self.factor = factor

    def __call__(self, weights):
        return self.factor * tf.reduce_sum(tf.abs(weights))

    def get_config(self):
        return {"factor": self.factor}
```

Remark 4. *There are some differences between a custom loss and a custom regularizer:*

- *we do not need to call the parent constructor, nor to merge the parent configuration;*
- *we implement the special method* `__call__` *instead of the method* `call`*.*

2.4 Metric wrapper

```python
import tensorflow as tf

class HuberMetric(tf.keras.metrics.Metric):
    def __init__(self, threshold=1.0, **kwargs):
        super().__init__(**kwargs)
        self.threshold = threshold
        # create_huber returns the Huber function of Exercise 1 for a given threshold
        self.huber_fn = create_huber(threshold)
        self.total = self.add_weight(name="total", shape=(), initializer="zeros")
        self.count = self.add_weight(name="count", shape=(), initializer="zeros")

    def update_state(self, y_true, y_pred, sample_weight=None):
        sample_metrics = self.huber_fn(y_true, y_pred)
        self.total.assign_add(tf.reduce_sum(sample_metrics))
        self.count.assign_add(tf.cast(tf.size(y_true), tf.float32))

    def result(self):
        return self.total / self.count

    def get_config(self):
        base_config = super().get_config()
        return {**base_config, "threshold": self.threshold}
```

Let us explain the above code:

- The constructor of `tf.keras.metrics.Metric` uses the method `add_weight` to create the variables that accumulate the metric over several batches.
- The method `update_state` updates these variables from the targets and the predictions of a single batch (sample weights are ignored here for simplicity).
- The method `result` returns the final value, here the mean Huber metric over all batches seen so far.
- The base class also provides the method `reset_state`, which resets all the variables to 0.0. It can be customized if necessary.

2.5 Custom layer with Keras

```python
import tensorflow as tf

class CustomDenseLayer(tf.keras.layers.Layer):
    def __init__(self, units=None, activation=None, **kwargs):
        super().__init__(**kwargs)
        self.units = units
        self.activation = tf.keras.activations.get(activation)

    def build(self, batch_input_shape):
        self.kernel = self.add_weight(name="kernel", shape=[batch_input_shape[-1], self.units],
        ↪  initializer="glorot_normal")
```

```
        self.bias = self.add_weight(name="bias", shape=[self.units], initializer="zeros")

    def call(self, X):
        return self.activation(X @ self.kernel + self.bias)

    def get_config(self):
        base_config = super().get_config()
        return {**base_config, "units": self.units, "activation": tf.keras.activations.serialize(self.activation)}
```

Exercise 2. *Implement a custom layer performing a normalization:*

- *The method* `build` *should create two trainable weights of type* `tf.float32`*, called* α *and* β*, respectively initialized to* 1 *and* 0*.*
- *The method* `call` *must compute the mean* μ *and the standard deviation* σ *of each instance, using the function* `tf.nn.moments` *with the arguments* `axes=-1` *and* `keepdims=True` *(it returns the mean and the variance). The output should be* $\alpha \otimes (X - \mu)/(\sigma + \epsilon) + \beta$*, where* $\otimes$ *is the pointwise multiplication and* $\epsilon = 0.001$*.*
- *Keras provides a similar built-in layer,* `tf.keras.layers.LayerNormalization`*. Check that your implementation produces approximately the same result. Why can the results differ slightly? (Hint: look at where* ϵ *is added in the built-in layer.)*

3 Optimizers

Training a neural network is fundamentally an optimization problem. Let W denote the collection of trainable parameters, viewed as a point in a Euclidean space. These parameters define a neural network, that is, a (typically nonlinear) mapping

$$f(\,\cdot\,; W) : \mathbb{R}^n \to \mathbb{R}^p.$$

The empirical risk associated with the parameter vector W is defined by

$$\mathscr{L}(W) = \frac{1}{N} \sum_{i=1}^{N} L(f(x_i; W),\, y_i)\,.$$

Given the network architecture, learning consists in determining the parameter vector W. To do so, two ingredients are required:

1. a training dataset

   $$\{(x_i, y_i)\}_{i=1}^{N},$$

   where $x_i \in \mathbb{R}^n$ is an input and y_i is the corresponding target;

2. a loss function

   $$L : \mathbb{R}^p \times \mathscr{Y} \to \mathbb{R},$$

   measuring the discrepancy between the network prediction and the target, where $\mathscr{Y}$ is the label space (e.g. $\{1, \dots, p\}$ for classification or $\mathbb{R}^p$ for regression).

The objective is then to find the parameter vector minimizing the empirical risk

$$W^* = \arg\min_{W} \mathscr{L}(W). \tag{1}$$

Remark 5. *In general, the optimization problem* (1) *is non-convex. Consequently, uniqueness of the minimizer is not guaranteed. In practice, optimization algorithms often converge to a local minimum or, more generally, to a stationary point.*

Exercise 3. *Consider the linear model*

$$f(x; W) = Wx, \qquad W \in \mathbb{R}^{p \times n},$$

and suppose that W^* minimizes the empirical hinge loss

$$\frac{1}{N} \sum_{i=1}^{N} \sum_{j \neq y_i} \max(0,\, (f(x_i; W))_j - (f(x_i; W))_{y_i} + 1)\,.$$

Is the minimizer W^* necessarily unique?

Since no closed-form solution is available in general, the parameters are computed using iterative optimization algorithms, commonly referred to as *optimizers*. The most fundamental approach is gradient descent, which updates the parameters using the local gradient of the empirical risk. Although effective, gradient descent may converge slowly, especially when the objective function is ill-conditioned or contains large flat regions. Over the years, numerous optimization algorithms have been proposed to improve convergence speed, stability, and robustness while retaining the simplicity of first-order methods.

Remark 6. *In deep learning, N is large and computing $\nabla_W \mathscr{L}(W_k)$ exactly at every iteration is too expensive. The gradient is instead estimated on a random mini-batch $B_k \subset \{1, \dots, N\}$,*

$$g_k = \frac{1}{|B_k|} \sum_{i \in B_k} \nabla_W L\big(f(x_i; W_k), y_i\big),$$

which is an unbiased estimator of $\nabla_W \mathscr{L}(W_k)$ when B_k is drawn uniformly at random. Gradient descent with this estimator is called stochastic gradient descent *(SGD). All the optimizers presented below are used in this stochastic setting: in their update rules, $\nabla_W \mathscr{L}(W_k)$ should be understood as the mini-batch estimate g_k.*

3.1 Momentum methods

The *momentum method* was introduced by Boris Polyak in 1964 as an improvement over standard gradient descent. The main idea is to augment the optimization process with a *momentum* vector that accumulates information from previous updates, thereby accelerating the optimization along directions where successive gradients are consistent.

Starting from an initial parameter vector and an initial momentum,

$$W_0,\ m_0 \in \mathbb{R}^d,$$

the iterations are defined by

$$\begin{aligned} m_{k+1} &= \beta\, m_k - \eta\, \nabla_W \mathscr{L}(W_k), \\ W_{k+1} &= W_k + m_{k+1}, \end{aligned}$$

where d denotes the number of trainable parameters, $\eta > 0$ is the learning rate, and $\beta \in [0, 1)$ is the *momentum coefficient*.

Remark 7. *The momentum vector can be interpreted as a velocity. At each iteration, it combines the current gradient with a fraction of the previous update. Consequently, if successive gradients point in approximately the same direction, the updates become progressively larger, leading to faster convergence. Conversely, when the gradient direction oscillates, the averaging effect of the momentum term damps these oscillations.*

Exercise 4. *Assume that the gradient remains constant throughout the optimization, i.e.,*

$$\nabla_W \mathscr{L}(W_k) = g$$

for every iteration.

Show that the momentum vector converges to

$$m_\infty = -\frac{\eta}{1-\beta} g.$$

Deduce that, for the commonly used value $\beta = 0.9$, the asymptotic update is ten times larger than that of standard gradient descent.

In 1983, Yurii Nesterov proposed an accelerated variant of the momentum method, called *Nesterov Accelerated Gradient (NAG)*, which evaluates the gradient at a look-ahead position obtained by moving in the current momentum direction. This leads to the following recursion:

$$\begin{aligned} m_{k+1} &= \beta\, m_k - \eta\, \nabla_W \mathscr{L}(W_k + \beta m_k), \\ W_{k+1} &= W_k + m_{k+1}. \end{aligned}$$

Remark 8. *Several deep learning libraries implement an algebraically equivalent formulation of Nesterov momentum whose update equations differ slightly from the original presentation. Although the recursions appear different, they produce the same optimization dynamics after a suitable change of variables.*

3.2 Adaptive optimizers

Unlike momentum-based methods, which accelerate the optimization using previous updates, *adaptive* optimization methods adjust the learning rate independently for each parameter. In the following, $\otimes$ and $\oslash$ denote the componentwise (Hadamard) product and division, and square roots of vectors are taken componentwise.

3.2.1 AdaGrad

The first method of this family, called *AdaGrad* (Adaptive Gradient), was proposed in 2011 by Duchi, Hazan, and Singer [3]. Its main idea is to assign a smaller learning rate to parameters that consistently experience large gradients, while allowing larger updates for parameters whose gradients remain small.

Starting from an initial accumulator $s_0 = 0$, the iterations are

$$\begin{aligned} s_{k+1} &= s_k + \nabla_W \mathscr{L}(W_k) \otimes \nabla_W \mathscr{L}(W_k), \\ W_{k+1} &= W_k - \eta \, \nabla_W \mathscr{L}(W_k) \oslash \sqrt{s_{k+1} + \epsilon}, \end{aligned}$$

where $\eta > 0$ is the initial learning rate and $\epsilon > 0$ is a small constant introduced for numerical stability.

Remark 9. *Parameters associated with consistently large gradients are updated more conservatively, whereas parameters whose gradients remain small continue to receive relatively large updates.*

3.2.2 RMSProp

Although AdaGrad adapts the learning rate to each parameter, it suffers from an important drawback. Since the accumulator

$$s_k = \sum_{i=0}^{k-1} \nabla_W \mathscr{L}(W_i) \otimes \nabla_W \mathscr{L}(W_i)$$

grows monotonically, the effective learning rate continuously decreases during training. After many iterations, the parameter updates may become extremely small, causing the optimization process to slow down or even nearly stop.

To overcome this limitation, Geoffrey Hinton proposed the *Root Mean Square Propagation (RMSProp)* optimizer. Instead of accumulating all past squared gradients, RMSProp computes an exponentially weighted moving average, thereby giving more importance to recent gradients while gradually forgetting older ones.

Starting from an initial accumulator $s_0 = 0$, the iterations are

$$\begin{aligned} s_{k+1} &= \beta \, s_k + (1 - \beta) \, \nabla_W \mathscr{L}(W_k) \otimes \nabla_W \mathscr{L}(W_k), \\ W_{k+1} &= W_k - \eta \, \nabla_W \mathscr{L}(W_k) \oslash \sqrt{s_{k+1} + \epsilon}, \end{aligned}$$

where $\beta \in [0, 1)$ is the decay coefficient, $\eta > 0$ is the learning rate, and all operations are performed componentwise.

Remark 10. *Unlike AdaGrad, the accumulator no longer grows indefinitely. Instead, it estimates the recent magnitude of the gradients, allowing the effective learning rates to remain stable throughout the optimization.*

3.2.3 Adam

Momentum methods and RMSProp improve gradient descent in two complementary ways. Momentum accelerates the optimization by accumulating previous updates, whereas RMSProp adapts the learning rate of each parameter according to the recent magnitude of its gradients.

The *Adaptive Moment Estimation (Adam)* optimizer, proposed by Kingma and Ba in 2014 [10], combines both ideas. It maintains two exponentially weighted moving averages:

- the first moment (the average gradient),
- the second moment (the average squared gradient).

Starting from

$$m_0 = 0, \qquad s_0 = 0,$$

the algorithm performs the iterations

$$\begin{aligned} m_{k+1} &= \beta_1\, m_k + (1-\beta_1)\, \nabla_W \mathscr{L}(W_k), \\ s_{k+1} &= \beta_2\, s_k + (1-\beta_2)\, \nabla_W \mathscr{L}(W_k) \otimes \nabla_W \mathscr{L}(W_k). \end{aligned}$$

Since both moving averages are initialized at zero, they are biased toward zero during the first iterations. Adam therefore introduces the bias-corrected estimates

$$\begin{aligned} \widehat{m}_{k+1} &= \frac{m_{k+1}}{1-\beta_1^{k+1}}, \\ \widehat{s}_{k+1} &= \frac{s_{k+1}}{1-\beta_2^{k+1}}. \end{aligned}$$

The parameter update is then

$$W_{k+1} = W_k - \eta\, \widehat{m}_{k+1} \oslash \sqrt{\widehat{s}_{k+1} + \epsilon}.$$

Typical values are

$$\beta_1 = 0.9, \qquad \beta_2 = 0.999, \qquad \epsilon = 10^{-8}.$$

Remark 11. *The first moment acts as a momentum term, while the second moment adapts the learning rate independently for each parameter. Adam is often fast and robust with little hyperparameter tuning, which explains why it has become one of the most widely used optimizers for deep neural networks. Faster convergence is nevertheless not guaranteed: Reddi et al. exhibited simple convex problems on which Adam fails to converge to the optimum [19], and adaptive methods can generalize worse than well-tuned SGD with momentum on some tasks [26].*

4 Regularization

Solving the optimization problem (1) with a well-chosen optimizer does not guarantee that the trained neural network captures general trends. Two problems may occur:

- *overfitting*: the network fits the training data too closely, including their noise, so that it predicts poorly on new data points. The fitted function then varies a lot with the particular training sample (high variance);
- *underfitting*: the network does not capture enough patterns from the training data, so that it performs poorly even on the training set (high bias).

Overfitting and underfitting are common when training deep neural networks. To detect them, one proceeds as follows:

1. split the available data into training, validation and test sets;
2. track the evolution of the loss (or of a metric) on both the training and the validation sets during training;
3. use the validation set to select hyperparameters (regularization strength, dropout rate, number of epochs, ...);
4. evaluate the final model *once* on the test set, which must never be used to make modelling choices.

The figure below shows how to identify these problems quickly:

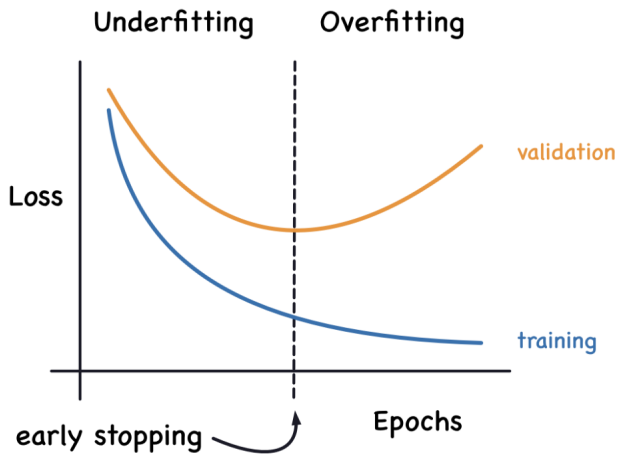


Figure 1: Evolution of the loss (or of a metric) on the training data (in blue) and on the validation data (in orange) during training.

Remark 12. *The best model is often obtained by stopping the training when the loss or metric on the validation data starts to increase. This is called the early stopping strategy. Since it limits how far the parameters move away from their initialization, early stopping is itself a form of implicit regularization.*

A *regularization* strategy aims at reducing overfitting. We present three main approaches:

1. adding a penalty term to the empirical risk (Section 4.1);
2. randomly perturbing the network during training, for instance with *dropout* (Section 4.2);
3. transforming the data or the activations before or during training (Section 4.3).

4.1 Regularization term

Adding a regularization term is the most common strategy for preventing overfitting. Instead of solving (1), we solve the *penalized* problem

$$\min_{W} \frac{1}{N} \sum_{i=1}^{N} L\big(f(x_i; W),\, y_i\big) \;+\; \lambda\, R(W)\,, \tag{2}$$

where $R(W) \geqslant 0$ is the regularization term and $\lambda \geqslant 0$ is a hyperparameter controlling its strength. The classical regularization terms are:

- the ℓ_1-regularizer: $R(W) = \|W\|_1 = \sum_j |w_j|$;
- the ℓ_2-regularizer: $R(W) = \|W\|_2^2 = \sum_j w_j^2$.

Remark 13. *The value of λ cannot be learned by minimizing (2) jointly in (W, λ): since $R \geqslant 0$, this would always lead to $\lambda = 0$. It is selected on the validation set, for instance by grid search.*

A closely related formulation is the *constrained* problem

$$\min_W \frac{1}{N} \sum_{i=1}^{N} L\big(f(x_i; W),\, y_i\big) \qquad \text{subject to} \qquad R(W) \leqslant C, \tag{3}$$

for a fixed $C > 0$. Only this second formulation explicitly bounds the weights by C. The two formulations are connected as follows.

Proposition 1. *Let $\mathscr{L}$ denote the empirical risk and let $\lambda \geqslant 0$. If W_λ is a minimizer of (2), then W_λ is a minimizer of the constrained problem (3) with $C = R(W_\lambda)$.*

Exercise 5. *Prove Proposition 1. (Hint: compare $\mathscr{L}(W) + \lambda R(W)$ and $\mathscr{L}(W_\lambda) + \lambda R(W_\lambda)$ for any W satisfying $R(W) \leqslant R(W_\lambda)$.)*

Remark 14. *Conversely, when $\mathscr{L}$ and R are convex and the constraint is qualified (for instance under Slater's condition), Lagrangian duality shows that every solution of the constrained problem (3) solves the penalized problem (2) for some multiplier $\lambda \geqslant 0$. For neural networks, $\mathscr{L}$ is not convex and this converse may fail; the two formulations should then be seen as two different, closely related, ways of restricting the effective size of the parameters.*

Let us visualize the constrained formulation in dimension 2, for the loss function $L : \mathbb{R}^2 \to \mathbb{R}$ defined by:

$$\forall\, w_1, w_2 \in \mathbb{R}\,, \quad L(w_1, w_2) = (w_1 - a)^2 + (w_2 - b)^2$$

where $a = 1.5$ and $b = 1$. The unconstrained minimum is reached at $(w_1, w_2) = (a, b)$, whose norm is greater than 1. Figure 2 shows the minimum obtained under the ℓ_1 and ℓ_2 constraints $\|W\|_1 \leqslant 1$ and $\|W\|_2 \leqslant 1$ (that is, $C = 1$).

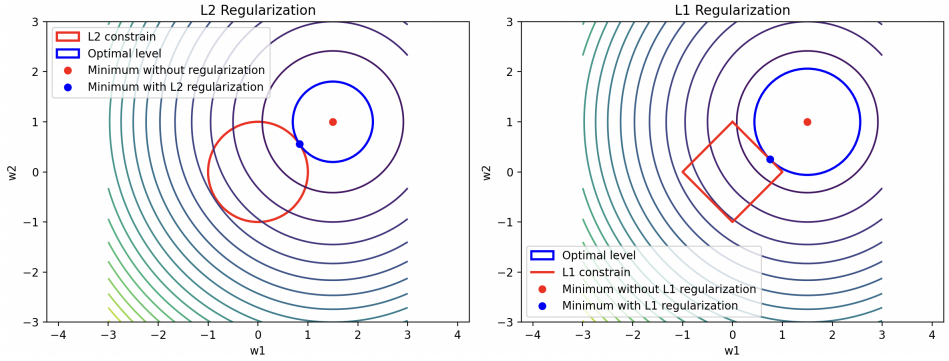


Figure 2: The contours illustrate the values of the loss function across different parameter values. The red point corresponds to the global minimum, whereas the blue point represents the minimum under the ℓ_2 (left) and ℓ_1 (right) constraints with $C = 1$.

Exercise 6. *In the setting of Figure 2, the constrained minimizer is the Euclidean projection of (a, b) onto the ball $\{\|W\| \leqslant 1\}$.*

1. *Compute the minimizers under the ℓ_2 and the ℓ_1 constraints for $(a, b) = (1.5, 1)$.*
2. *Same question for $(a, b) = (1.5, 0.2)$.*
3. *What do you observe for the ℓ_1 constraint? Explain geometrically why ℓ_1 regularization tends to produce* sparse *solutions, i.e. solutions with exact zeros.*

Remark 15. *The penalized problem has a Bayesian interpretation. If the targets are modelled with a likelihood proportional to $\exp(-L)$ and the weights are given a prior density $\pi(W)$, then the maximum a posteriori estimator minimizes $\sum_i L(f(x_i; W), y_i) - \log \pi(W)$. A Gaussian prior $\mathcal{N}(0, \sigma^2 I)$ yields the ℓ_2 penalty, and a Laplace prior yields the ℓ_1 penalty.*

Remark 16. *For plain gradient descent, the ℓ_2 penalty $\frac{\lambda}{2}\|W\|_2^2$ leads to the update*

$$W_{k+1} = (1 - \eta\lambda)\, W_k - \eta\, \nabla_W \mathscr{L}(W_k),$$

which shrinks the weights by a constant factor at each step. This is why ℓ_2 regularization is also called weight decay *(see the training of AlexNet in Section 5.6.2). For adaptive optimizers such as Adam, adding the ℓ_2 penalty to the loss and decaying the weights directly are no longer equivalent; the latter variant is called AdamW [15]. In Keras, a penalty can be attached to the weights of a layer through its* `kernel_regularizer` *argument, for instance* `tf.keras.regularizers.L2(1e-4)`*.*

4.2 Dropout

Dropout [23] randomly deactivates a fixed proportion of the *neurons* (that is, of the activations) of selected layers at each training step. The weights themselves are not removed: dropout perturbs the network during training, but does not reduce its number of parameters.

Formally, let $h \in \mathbb{R}^m$ be the output of a layer and let $p \in [0, 1)$ be the *dropout rate*. At each training step, one draws independent random variables

$$\xi_1, \dots, \xi_m \sim \text{Bernoulli}(1 - p)$$

and replaces h by

$$\widetilde{h} = \frac{1}{1 - p}\, \xi \otimes h.$$

The factor $1/(1 - p)$ (*inverted dropout*, as implemented in Keras) ensures that $\mathbb{E}[\widetilde{h} \mid h] = h$. At inference time, dropout is disabled and $\widetilde{h} = h$.

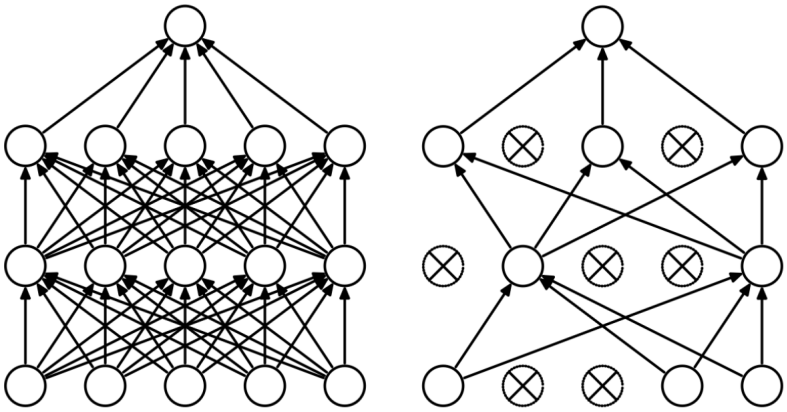


Figure 3: Example of dropout applied to all layers, except the last one.

Remark 17. *A new mask ξ is drawn at every training step: a neuron deactivated at one step may be active at the next one.*

The idea behind dropout is to prevent the network from relying too much on a few neurons, which may specialize in specific patterns of the training inputs. Each training step updates a random sub-network, and the final network can be interpreted as an approximate average of this exponentially large ensemble of sub-networks [23].

Exercise 7. *1. Show that $\mathbb{E}[\widetilde{h}_j \mid h] = h_j$ and $\text{Var}(\widetilde{h}_j \mid h) = \frac{p}{1-p}\, h_j^2$.*

2. Consider a linear regression model $\hat{y} = w^\top x$ and apply dropout to the input $x \in \mathbb{R}^m$. Show that

$$\mathbb{E}\big[(y - w^\top \widetilde{x})^2\big] = (y - w^\top x)^2 + \frac{p}{1 - p} \sum_{j=1}^{m} x_j^2\, w_j^2.$$

Deduce that, in expectation, dropout acts as a data-dependent ℓ_2 penalty.

Let us illustrate the effect of dropout on a simple model which classifies handwritten digits from the MNIST dataset. The original training set is split into a training set (50,000 images) and a validation set (10,000 images); the test set is only used for the final evaluation.

```python
import os

import cv2

import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf
import tensorflow_datasets as tfds

# Indicate a path to store your models and data

MAIN_PATH = "./dropout"

# Creation of necessary subfolders

subfolders = ["data", "models", "figures"]
for subfolder in subfolders:
    os.makedirs(f"{MAIN_PATH}/{subfolder}", exist_ok=True)


# Some utils


def normalize_img(image, label):
    """Normalizes images: `uint8` -> `float32`."""
    return tf.squeeze(tf.cast(image, tf.float32), axis=-1) / 255.0, label


def plot_losses(nb_epochs, loss, val_loss, path_to_save):
    epochs = range(1, nb_epochs + 1)

    plt.figure()

    plt.plot(epochs, loss, "o-", label="Training loss")
    plt.plot(epochs, val_loss, "o-", label="Validation loss")
    plt.title("Training and validation loss")
    plt.xlabel("Epoch")
    plt.legend()

    plt.savefig(path_to_save)
    plt.close()


def save_test_images(ds_test, path_to_save):
    for images, labels in ds_test:
        _images = images.numpy()
        _labels = labels.numpy()

        np.save(f"{path_to_save}/labels.npy", _labels)

        for i in range(_images.shape[0]):
            _image = _images[i, :, :].reshape(28, 28)
            cv2.imwrite(f"{path_to_save}/image_{i}.png", _image)

        break


# Load the dataset: 50,000 training images, 10,000 validation images, 10,000 test images

(ds_train, ds_val, ds_test), ds_info = tfds.load(
    "mnist",
    split=["train[:50000]", "train[50000:]", "test"],
    shuffle_files=True,
    as_supervised=True,
    with_info=True,
)

ds_train = ds_train.map(normalize_img, num_parallel_calls=tf.data.AUTOTUNE)
ds_train = ds_train.shuffle(10_000).batch(32).prefetch(tf.data.AUTOTUNE)

ds_val = ds_val.map(normalize_img, num_parallel_calls=tf.data.AUTOTUNE)
```

```
ds_val = ds_val.batch(32).prefetch(tf.data.AUTOTUNE)

save_test_images(
    ds_test=ds_test.batch(32).prefetch(tf.data.AUTOTUNE),
    path_to_save=f"{MAIN_PATH}/data",
)

ds_test = ds_test.map(normalize_img, num_parallel_calls=tf.data.AUTOTUNE)
ds_test = ds_test.batch(32).prefetch(tf.data.AUTOTUNE)

# Build the models respectively without and with the dropout

model2_without_dropout = tf.keras.Sequential(
    [
        tf.keras.Input(shape=(28, 28)),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(100, activation="relu"),
        tf.keras.layers.Dense(100, activation="relu"),
        tf.keras.layers.Dense(10, activation="softmax"),
    ]
)

model2_with_dropout = tf.keras.Sequential(
    [
        tf.keras.Input(shape=(28, 28)),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dropout(rate=0.2),
        tf.keras.layers.Dense(100, activation="relu"),
        tf.keras.layers.Dropout(rate=0.2),
        tf.keras.layers.Dense(100, activation="relu"),
        tf.keras.layers.Dropout(rate=0.2),
        tf.keras.layers.Dense(10, activation="softmax"),
    ]
)

# Indicate the training parameters

EPOCHS = 25
LR = 0.001


# Compile the models

model2_without_dropout.compile(
    optimizer=tf.keras.optimizers.Adam(LR),
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
    metrics=[tf.keras.metrics.SparseCategoricalAccuracy()],
)

model2_with_dropout.compile(
    optimizer=tf.keras.optimizers.Adam(LR),
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
    metrics=[tf.keras.metrics.SparseCategoricalAccuracy()],
)

# Train the models (the validation set, not the test set, is used to monitor training)

history_without_dropout = model2_without_dropout.fit(
    ds_train, validation_data=ds_val, epochs=EPOCHS
)
history_with_dropout = model2_with_dropout.fit(
    ds_train, validation_data=ds_val, epochs=EPOCHS
)

# Final evaluation on the test set

model2_without_dropout.evaluate(ds_test)
model2_with_dropout.evaluate(ds_test)

# Save the models

model2_without_dropout.save(f"{MAIN_PATH}/models/model_without_dropout.keras")
model2_with_dropout.save(f"{MAIN_PATH}/models/model_with_dropout.keras")
```

```
# Retrieve the losses computed during the training step

losses_training_without_dropout = history_without_dropout.history["loss"]
losses_validation_without_dropout = history_without_dropout.history["val_loss"]
losses_training_with_dropout = history_with_dropout.history["loss"]
losses_validation_with_dropout = history_with_dropout.history["val_loss"]

# Save the evolution of losses

plot_losses(
    nb_epochs=EPOCHS,
    loss=losses_training_without_dropout,
    val_loss=losses_validation_without_dropout,
    path_to_save=f"{MAIN_PATH}/figures/loss_without_dropout.png",
)

plot_losses(
    nb_epochs=EPOCHS,
    loss=losses_training_with_dropout,
    val_loss=losses_validation_with_dropout,
    path_to_save=f"{MAIN_PATH}/figures/loss_with_dropout.png",
)
```

Exercise 8. *After running the above code, answer the following questions:*

- *Compare the two generated plots,* ***loss_without_dropout.png*** *and* ***loss_with_dropout.png****. What is the main difference?*
- *Why can the training loss of the model with dropout be higher than its validation loss?*
- *What do you conclude?*

4.3 Data augmentation and normalization

Another common way of inducing implicit regularization is to transform the data or the intermediate activations. Two common approaches are:

- **Data augmentation:** this technique artificially increases the size of the training set by creating modified versions of existing data points. In computer vision, for example, transformations such as small translations, rotations, flips, or color shifts help neural networks learn more robust representations.
- **Normalization layers:** these layers standardize activations by subtracting a mean and dividing by a standard deviation. Normalization improves numerical stability and often accelerates training.

More formally, let $\mathscr{G}$ be a family of transformations which should not change the label (for instance small translations of an image), and let ν be a probability distribution on $\mathscr{G}$. Data augmentation replaces the empirical risk by

$$\frac{1}{N}\sum_{i=1}^{N}\mathbb{E}_{g\sim\nu}\Big[L\big(f(g\cdot x_i;W),y_i\big)\Big],$$

which is estimated by drawing a new transformation g each time an example is used. This encourages the network to be approximately *invariant* to the transformations in $\mathscr{G}$, in the sense of Section 5.2.4. We will explore data augmentation in more detail during the exercise sessions, where you will implement and visualize several augmentation strategies and study their impact on model performance.

In the case of Batch Normalization [8], for a mini-batch of activations $\{x_1,\dots,x_m\}$ of a given feature (or, in a CNN, of a given channel), the normalized output is computed as:

$$\hat{x}_i=\frac{x_i-\mu_B}{\sqrt{\sigma_B^2+\varepsilon}},$$

where μ_B and σ_B^2 are the mean and variance computed over the mini-batch, and ε is a small constant to ensure numerical stability. The normalized values are then scaled and shifted using learnable parameters γ and β:

$$y_i=\gamma\hat{x}_i+\beta.$$

This adaptive re-scaling allows the network to preserve its representational capacity while benefiting from the normalization effect. At inference time, the mini-batch statistics are replaced by running averages of μ_B and σ_B^2 accumulated during training, so that the layer becomes a fixed affine map and the prediction for an example no longer depends on the other examples of the batch.

Remark 18. *Batch Normalization was originally motivated by the reduction of the internal covariate shift, i.e. the change in the distribution of the inputs of a layer during training as the parameters of the previous layers are updated [8]. This explanation has since been questioned: Santurkar et al. [20] argue that the main benefit of Batch Normalization is to make the optimization landscape smoother. Its regularizing effect comes mostly from the noise introduced by the mini-batch statistics. Transformers use instead Layer Normalization (Section 6.6), which normalizes each token independently of the rest of the batch.*

5 Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs) were originally designed to automatically extract spatial features from images. An image can be represented as a matrix or tensor, where each element corresponds to a pixel. In grayscale images, each pixel has an intensity value between 0 and 255, with 255 representing white. For color images, each pixel is typically represented as a triplet (r, g, b), where r, g, and b denote the intensity of the red, green, and blue channels, respectively, each ranging from 0 to 255.

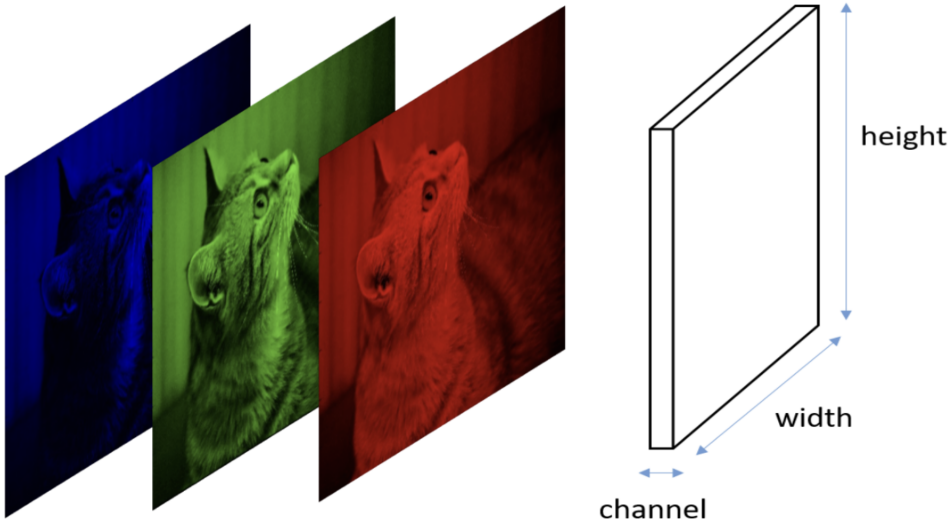


Figure 4: A color image can be represented as the superposition of three channels (red, green and blue) each encoding the intensity of its respective color component.

Training a classical Multi-Layer Perceptron (MLP) directly on raw image data is inefficient due to the extremely high number of parameters required. For instance, an image of size 100×100 contains 10,000 pixels; if the first hidden layer of an MLP has 1,000 neurons, this results in 10 million connections—an impractically large number that can easily lead to overfitting and high computational cost.

CNNs address this issue by introducing local connectivity and weight sharing. Instead of connecting every neuron to every pixel, convolutional layers apply a set of small filters (also called kernels) that slide across the image. Each filter learns to detect specific local patterns such as edges or textures, enabling the network to capture hierarchical spatial features efficiently.

5.1 Convolution product

CNNs are particularly effective for extracting features from data with a grid-like topology, such as images or time series. Before delving deeper into the architecture of these networks, let us first recall the definition of the convolution operation for a 1-dimensional signal.

Let $f, g : \mathbb{R} \to \mathbb{R}$ be two integrable functions, i.e. $f, g \in L^1(\mathbb{R})$. The convolution of f and g, denoted by $f * g$, is defined for almost every $t \in \mathbb{R}$ as:

$$(f * g)(t) = \int_{-\infty}^{+\infty} f(t-s)\, g(s)\, ds = \int_{-\infty}^{+\infty} f(s)\, g(t-s)\, ds,$$

and, by Fubini's theorem, $f * g \in L^1(\mathbb{R})$.

Intuitively, when $g \geqslant 0$ and $\int g = 1$, the convolution can be interpreted as a **weighted average** of the function f around each point t, where the weights are provided by the function g. In this context, g is referred to as the **kernel** or **filter** of the convolution.

Remark 19. *The convolution product $*$ satisfies the following properties:*

- ***Commutativity.*** $f * g = g * f$
- ***Associativity.*** $f * (g * h) = (f * g) * h$
- ***Bilinearity.*** $\lambda(f * g) = (\lambda f) * g = f * (\lambda g)$ *and* $(f + g) * h = f * h + g * h$

where f, g and h belong to $L^1(\mathbb{R})$, and λ is a real number.

In practice, we usually deal with discrete signals, as is the case for grayscale images, which can be represented as two-dimensional arrays. If we now consider two discrete functions $f, g : \mathbb{Z} \to \mathbb{R}$ (for instance with finite support), their discrete convolution is defined as:

$$(f * g)[n] = \sum_{k=-\infty}^{+\infty} f[n-k]\, g[k] = \sum_{k=-\infty}^{+\infty} f[k]\, g[n-k]$$

Let I be a grayscale image of dimensions $w \times h$, and let K be a kernel (or filter) of odd dimensions $k_w \times k_h$, with $k_w = 2p + 1$ and $k_h = 2q + 1$. It is convenient to index the kernel around its center, so that these can be viewed respectively as mappings

$$I : [\![1, w]\!] \times [\![1, h]\!] \to \mathbb{R} \quad \text{and} \quad K : [\![-p, p]\!] \times [\![-q, q]\!] \to \mathbb{R},$$

where I is extended by zero outside its domain. The two-dimensional discrete convolution between I and K is then defined by:

$$(I * K)[m, n] = \sum_{i=-p}^{p} \sum_{j=-q}^{q} I[m-i, n-j]\, K[i, j].$$

Remark 20. *In image processing, the result of such an operation is often called a filtered image, and the matrix K is referred to as a filter or kernel. Filters are widely used for tasks such as denoising, edge detection, and texture extraction.*

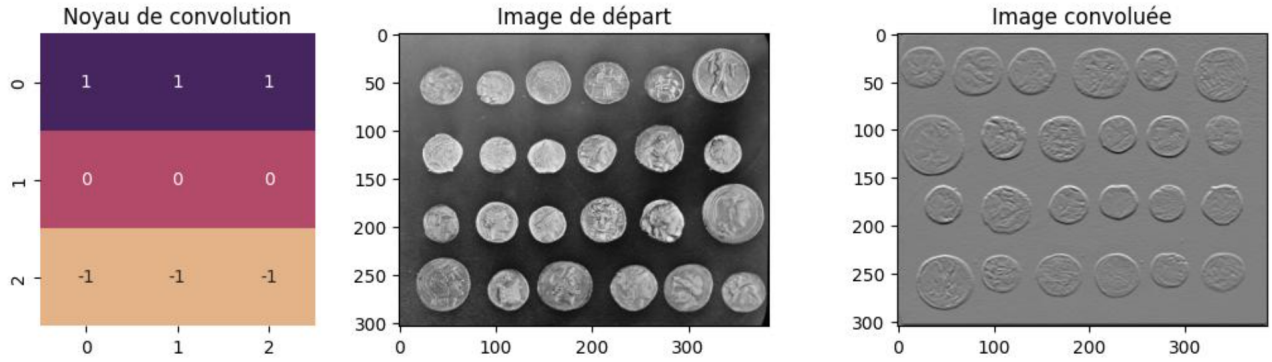


Figure 5: Example of convolution product between an image and a Prewitt filter

Remark 21. *In practice, most deep learning libraries implement the cross-correlation operation instead of the mathematical convolution, which avoids flipping the kernel. The cross-correlation between I and K is the matrix $I \star K$ defined by:*

$$(I \star K)[m, n] = \sum_{i=-p}^{p} \sum_{j=-q}^{q} I[m+i, n+j]\, K[i, j].$$

Unlike convolution, cross-correlation does not involve flipping the kernel, and thus the operation is not commutative. However, this distinction has no practical consequence for convolutional neural networks, since the kernel weights are learned during training. As a result, the terms convolution and cross-correlation are often used interchangeably in the deep learning literature. All the formulas below are written as cross-correlations.

5.2 Convolution layer

Roughly speaking, a *convolutional layer* applies a local linear transformation to an input tensor using a collection of learnable filters, also called *kernels*. The same filters are applied at every spatial location, which results in a sharing of the parameters across the input.

An image can naturally be represented as a tensor. A color image, for example, can be represented by a tensor of shape (H, W, C), where H and W denote respectively the height and the width of the image, and C denotes the number of channels. For a grayscale image, $C = 1$, whereas for a standard RGB image, $C = 3$.

Two important hyperparameters determine how the kernels are applied spatially:

- **Padding:** the addition of values, usually zeros, around the spatial boundary of the input tensor before applying the kernels. Padding allows the spatial dimensions of the output to be controlled and, in particular, can be chosen so that they are preserved.
- **Stride:** the number of positions by which the kernel is shifted between two consecutive applications. Increasing the stride generally decreases the spatial dimensions of the output and therefore introduces a downsampling effect.

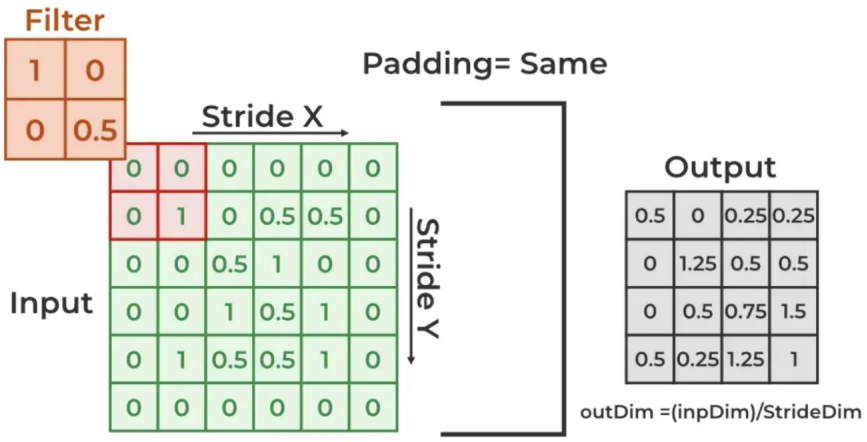


Figure 6: Illustration of padding and stride.

Remark 22. *The entries of the kernels are learnable parameters. They are therefore updated during training by backpropagation, together with the other parameters of the network.*

We can now give a formal definition of a convolutional layer.

Definition 1. *Let X be an input tensor of shape (H, W, C_{in}), and let a convolutional layer contain C_{out} learnable kernels, each of spatial dimensions (k_h, k_w).*

For $r \in \{1, \dots, C_{\text{out}}\}$, let

$$K^{(r)} \in \mathbb{R}^{k_h \times k_w \times C_{\text{in}}}$$

denote the r-th kernel and let $b_r \in \mathbb{R}$ be its associated bias.

For simplicity, assume that the kernels have odd spatial dimensions, define

$$p = \frac{k_h - 1}{2}, \qquad q = \frac{k_w - 1}{2},$$

and index the entries $k^{(r)}_{u,v,\ell}$ of $K^{(r)}$ by $u \in [\![-p, p]\!]$, $v \in [\![-q, q]\!]$ and $\ell \in [\![1, C_{\text{in}}]\!]$. For vertical and horizontal strides s_h and s_w, the convolutional layer produces an output tensor

$$Z \in \mathbb{R}^{H_{\text{out}} \times W_{\text{out}} \times C_{\text{out}}},$$

whose entries are given by

$$z_{i,j,r} = b_r + \sum_{u=-p}^{p} \sum_{v=-q}^{q} \sum_{\ell=1}^{C_{\text{in}}} x_{is_h+u,\, js_w+v,\, \ell}\, k^{(r)}_{u,v,\ell}.$$

Here, r indexes the output feature map, while ℓ indexes the input channel. The range of the output indices (i, j) depends on the padding (see Exercise 9).

Remark 23. *Each kernel $K^{(r)}$ contains $k_h k_w C_{\text{in}}$ learnable weights and one bias b_r. Consequently, a convolutional layer with C_{out} kernels contains*

$$C_{\text{out}} \left(k_h k_w C_{\text{in}} + 1\right)$$

learnable parameters.

5.2.1 Matrix representation of a convolution

The previous definition also reveals an important connection between convolutional layers and linear algebra. Before applying the bias and a possible nonlinear activation function, a convolution is a linear transformation. Consequently, it can be represented as a matrix-vector product.

To keep the notation simple, consider a one-dimensional input signal with a single channel,

$$x = \begin{pmatrix} x_1 & x_2 & \cdots & x_n \end{pmatrix}^\top \in \mathbb{R}^n,$$

and a kernel

$$h = \begin{pmatrix} h_1 & h_2 & \cdots & h_k \end{pmatrix}^\top \in \mathbb{R}^k, \qquad k \leqslant n.$$

With unit stride and no padding, the convolutional layer (in its cross-correlation form) produces the vector $y \in \mathbb{R}^{n-k+1}$ defined by

$$y_i = \sum_{j=1}^{k} h_j x_{i+j-1}, \qquad i = 1, \ldots, n-k+1.$$

For example, if $k = 3$, the successive output values are

$$\begin{aligned} y_1 &= h_1 x_1 + h_2 x_2 + h_3 x_3, \\ y_2 &= h_1 x_2 + h_2 x_3 + h_3 x_4, \\ &\vdots \\ y_{n-2} &= h_1 x_{n-2} + h_2 x_{n-1} + h_3 x_n. \end{aligned}$$

Hence, the convolution can be represented as

$$y = T_h x, \qquad T_h = \begin{pmatrix} h_1 & h_2 & \cdots & h_k & 0 & \cdots & 0 \\ 0 & h_1 & h_2 & \cdots & h_k & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & & \ddots & 0 \\ 0 & \cdots & 0 & h_1 & h_2 & \cdots & h_k \end{pmatrix} \in \mathbb{R}^{(n-k+1)\times n},$$

which is a sparse structured matrix.

In particular, T_h has a Toeplitz structure: its coefficients are constant along diagonals, with each row corresponding to a shifted application of the same kernel.

Remark 24. *The matrix representation makes explicit two fundamental properties of convolutional layers.*

- ***Local connectivity:*** *each row of T_h contains at most k non-zero coefficients. Therefore, each output value depends only on a local neighborhood of the input.*
- ***Weight sharing:*** *the same coefficients $h_1, \ldots, h_k$ appear repeatedly in the different rows of T_h. Thus, the same transformation is applied at every spatial position.*

Exercise 9. *Consider a convolutional layer with kernel size (k_h, k_w), vertical stride s_h, horizontal stride s_w, and no padding.*

1. *Show that the spatial dimensions of the output tensor are*

$$\left(\left\lfloor \frac{H - k_h}{s_h} \right\rfloor + 1, \left\lfloor \frac{W - k_w}{s_w} \right\rfloor + 1 \right).$$

2. *Assume now that a zero-padding of size P is applied to all four borders of the input tensor. Show that the output dimensions become*
$$\left(\left\lfloor \frac{H+2P-k_h}{s_h} \right\rfloor + 1, \left\lfloor \frac{W+2P-k_w}{s_w} \right\rfloor + 1\right).$$

3. *Assume that $s_h = s_w = 1$ and that both k_h and k_w are odd integers. Determine the padding size P for which the output tensor has the same spatial dimensions as the input tensor.*

4. *What is the total number of parameters to train when applying a single convolution layer as in Definition 1?*

5.2.2 Translation equivariance

One of the fundamental consequences of weight sharing is that a convolutional layer applies the same local transformation independently of the spatial position. As a consequence, convolutional layers possess an important symmetry property called *translation equivariance*. To formalize this notion, we first introduce the translation operator.

Definition 2. *Let $X : \mathbb{Z}^2 \longrightarrow \mathbb{R}^C$ be a discrete multi-channel signal, and let $\tau = (\tau_h, \tau_w) \in \mathbb{Z}^2$ be a translation vector. The translation operator T_τ is defined by*

$$(T_\tau X)[i, j, c] = X[i - \tau_h, j - \tau_w, c],$$

for every spatial position (i, j) and every channel c.

Thus, the translation operator acts only on the spatial coordinates and leaves the channel index unchanged.

Definition 3. *Let Φ be an operator acting on spatial signals. We say that Φ is translation equivariant if, for every input X and every translation vector τ, one has*

$$\Phi(T_\tau X) = T_\tau \Phi(X).$$

In other words, translating the input before applying Φ produces the same result as applying Φ first and translating its output by the same amount. We can now show that a convolutional layer with stride equal to one satisfies this property.

Proposition 2. *Consider a convolutional layer with stride 1, containing C_{out} kernels*

$$K^{(r)} \in \mathbb{R}^{k_h \times k_w \times C_{\text{in}}}, \qquad r \in \{1, \dots, C_{\text{out}}\}.$$

Assume that the input is defined on $\mathbb{Z}^2$, so that boundary effects can be ignored. Let Φ denote the corresponding convolutional operator defined by

$$\Phi(X)[i, j, r] = b_r + \sum_{u=-p}^{p} \sum_{v=-q}^{q} \sum_{\ell=1}^{C_{\text{in}}} X[i + u, j + v, \ell]\, k^{(r)}_{u,v,\ell},$$

where

$$p = \frac{k_h - 1}{2}, \qquad q = \frac{k_w - 1}{2}.$$

Then Φ is translation equivariant, that is,

$$\Phi(T_\tau X) = T_\tau \Phi(X)$$

for every $\tau \in \mathbb{Z}^2$.

Proof. Let

$$\tau = (\tau_h, \tau_w) \in \mathbb{Z}^2.$$

For every output position (i, j) and every output channel r, we have

$$\begin{aligned}
\Phi(T_\tau X)[i, j, r] &= b_r + \sum_{u=-p}^{p} \sum_{v=-q}^{q} \sum_{\ell=1}^{C_{\text{in}}} (T_\tau X)[i + u, j + v, \ell]\, k^{(r)}_{u,v,\ell} \\
&= b_r + \sum_{u=-p}^{p} \sum_{v=-q}^{q} \sum_{\ell=1}^{C_{\text{in}}} X[i + u - \tau_h, j + v - \tau_w, \ell]\, k^{(r)}_{u,v,\ell}.
\end{aligned}$$

By definition of Φ, the last expression is equal to

$$\Phi(X)[i - \tau_h, j - \tau_w, r].$$

Using the definition of the translation operator on the output feature map, we obtain

$$\Phi(X)[i - \tau_h, j - \tau_w, r] = (T_\tau \Phi(X))[i, j, r].$$

Consequently,

$$\Phi(T_\tau X) = T_\tau \Phi(X),$$

which proves the translation equivariance of the convolutional layer. □

This property has an important interpretation in the context of feature extraction. Suppose that a kernel has learned to detect a particular local pattern, such as a vertical edge. If this pattern is translated within the input, the corresponding activation in the output feature map is translated by the same amount. Therefore, the kernel can detect the same local structure independently of its absolute spatial position.

Remark 25. *The addition of a bias does not affect translation equivariance because the same scalar b_r is added at every spatial position of a given output channel.*

5.2.3 Pointwise activation functions

A convolutional layer is generally followed by a pointwise nonlinear activation function. Translation equivariance is preserved by such operations.

Let $\sigma : \mathbb{R} \longrightarrow \mathbb{R}$ be an activation function acting independently on every component of a feature map, and define

$$\Psi(X) = \sigma\left(\Phi(X)\right).$$

Since Φ is translation equivariant, we have

$$\begin{aligned} \Psi(T_\tau X) &= \sigma\left(\Phi(T_\tau X)\right) \\ &= \sigma\left(T_\tau \Phi(X)\right) \\ &= T_\tau \sigma\left(\Phi(X)\right) \\ &= T_\tau \Psi(X). \end{aligned}$$

Thus, the composition of a stride-one convolution and a pointwise activation function remains translation equivariant.

Remark 26. *More generally, a composition of translation-equivariant operators remains translation equivariant. Consequently, a sequence of stride-one convolutional layers and pointwise activation functions preserves translation equivariance, as long as boundary effects and spatial subsampling are ignored.*

5.2.4 Equivariance and invariance

It is important to distinguish *equivariance* from *invariance*. An operator Φ is translation equivariant if

$$\Phi(T_\tau X) = T_\tau \Phi(X),$$

whereas an operator Φ is translation invariant if

$$\Phi(T_\tau X) = \Phi(X).$$

The distinction is fundamental. Translation equivariance means that a transformation of the input induces a corresponding transformation of the output. Translation invariance means that the output does not change when the input is translated. A convolutional feature extractor is naturally translation equivariant. For example, if a feature is shifted by five pixels in the input, its corresponding activation is also shifted by five pixels in the feature map. For a classification problem, however, one generally expects the final prediction to be largely independent of the precise location of the object. Spatial aggregation operations, such as pooling or global average pooling, can therefore be used to progressively reduce the dependence of the final representation on the spatial position of the extracted features.

Remark 27. *Translation equivariance should not be confused with translation invariance. A convolutional layer preserves information about the spatial location of a feature, whereas a translation-invariant representation does not depend on that location.*

5.2.5 Effect of the stride

The previous proposition assumes that the stride is equal to one. When the stride is greater than one, the convolutional layer also performs spatial subsampling, and exact equivariance with respect to arbitrary pixel translations is generally lost. Consider a convolutional layer with vertical and horizontal strides s_h and s_w.

Its output is given by

$$\Phi_s(X)[i,j,r] = b_r + \sum_{u=-p}^{p} \sum_{v=-q}^{q} \sum_{\ell=1}^{C_{\text{in}}} X[is_h + u, js_w + v, \ell]\, k^{(r)}_{u,v,\ell}.$$

Let $\delta = (\delta_h, \delta_w) \in \mathbb{Z}^2$ and consider the input translation $\tau = (s_h\delta_h, s_w\delta_w)$. Then

$$\begin{aligned}\Phi_s(T_\tau X)[i,j,r] &= b_r + \sum_{u=-p}^{p} \sum_{v=-q}^{q} \sum_{\ell=1}^{C_{\text{in}}} X[is_h + u - s_h\delta_h, js_w + v - s_w\delta_w, \ell]\, k^{(r)}_{u,v,\ell} \\ &= \Phi_s(X)[i - \delta_h, j - \delta_w, r].\end{aligned}$$

Therefore,

$$\Phi_s\left(T_{(s_h\delta_h, s_w\delta_w)} X\right) = T_\delta \Phi_s(X).$$

Hence, a strided convolution remains equivariant to translations that are compatible with its sampling grid. An input translation of s_h pixels vertically and s_w pixels horizontally corresponds to a translation of one position in the output feature map. For translations that are not multiples of the stride, no such exact relation holds in general.

Remark 28. *This explains why strided convolutions and pooling layers can reduce translation equivariance. Since these operations retain only a subset of the spatial positions, their outputs depend on the alignment of the input with the sampling grid.*

5.3 Pooling and Unpooling layers

Convolutional layers preserve the spatial structure of their input, although the spatial dimensions can be reduced by using a stride greater than one. Another common way of reducing the spatial resolution of feature maps is to use a *pooling layer*.

Pooling layers are local operations that aggregate the values contained in a small spatial neighborhood of a feature map. Their main objectives are to reduce the spatial dimensions of the representation, decrease the computational and memory requirements of subsequent layers, and introduce a degree of robustness to small spatial translations of the input.

Unlike convolutional layers, pooling layers do not contain learnable parameters. The aggregation rule is fixed in advance.

5.3.1 Max and average pooling

Let

$$X \in \mathbb{R}^{H \times W}$$

be a feature map. Consider a pooling window of spatial dimensions $k_h \times k_w$ and vertical and horizontal strides s_h and s_w. The pooled feature map $Z = (z_{i,j})$ is defined by:

- Max pooling:

$$z_{i,j} = \max_{0 \leqslant u < k_h,\, 0 \leqslant v < k_w} x_{is_h+u,\, js_w+v}.$$

- Average pooling:

$$z_{i,j} = \frac{1}{k_h k_w} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} x_{is_h+u,\, js_w+v}.$$

Here the output indices start at $i = j = 0$ and the input indices at 0. Both operations are applied independently to each channel of a feature map. Pooling therefore generally reduces the spatial dimensions while preserving the number of channels; without padding, the output dimensions are given by the same formula as for a convolution (Exercise 9).

Remark 29. *Max pooling and average pooling differ in the type of information that they preserve. Max pooling retains the largest activation in each local region and can therefore be interpreted as detecting whether a strong response to a particular feature is present. Average pooling instead summarizes the average level of activation in the region.*

5.3.2 Unpooling

Pooling is a many-to-one operation: several input values are aggregated into a single output value. Consequently, the original input cannot, in general, be uniquely reconstructed from the pooled representation. An operation that increases the spatial resolution of a feature map is generally referred to as *upsampling*. The term *unpooling* is used for specific upsampling operations associated with pooling layers.

A simple upsampling operation consists in increasing the spatial dimensions of a feature map by replicating its values. Other upsampling methods use interpolation, such as nearest-neighbor or bilinear interpolation. These operations do not involve learnable parameters.

In what follows, we assume non-overlapping pooling windows, i.e. $s_h = k_h$ and $s_w = k_w$, and denote by $\widetilde{X} \in \mathbb{R}^{H \times W}$ the unpooled feature map.

Average unpooling. The simplest unpooling operation copies each pooled value to every position of its window:

$$\widetilde{x}_{is_h+u,\, js_w+v} = z_{i,j}, \qquad 0 \leqslant u < k_h,\ 0 \leqslant v < k_w.$$

Max-unpooling. A particular form of unpooling associated with max pooling is called *max-unpooling*. During max pooling, one can record the position at which the maximum value is attained. This information can then be used to place the pooled value back at the corresponding position during the unpooling operation.

Let $z_{i,j}$ be the output of max pooling,

$$z_{i,j} = \max_{0 \leqslant u < k_h,\, 0 \leqslant v < k_w} x_{is_h+u,\, js_w+v}.$$

In addition to the maximum value, we record the position at which the maximum is attained:

$$(u_{i,j}, v_{i,j}) \in \operatorname*{argmax}_{0 \leqslant u < k_h,\, 0 \leqslant v < k_w} x_{is_h+u,\, js_w+v}.$$

The pair

$$(z_{i,j}, (u_{i,j}, v_{i,j}))$$

therefore contains the maximum value together with its position inside the corresponding pooling window. The max-unpooled feature map is then defined by

$$\widetilde{x}_{is_h+u,\, js_w+v} = \begin{cases} z_{i,j}, & \text{if } (u,v) = (u_{i,j}, v_{i,j}), \\ 0, & \text{otherwise,} \end{cases} \qquad 0 \leqslant u < k_h,\ 0 \leqslant v < k_w.$$

Remark 30. *Max-unpooling is not the inverse of max pooling in the mathematical sense. Indeed, max pooling is not an injective operation: different input feature maps can produce exactly the same pooled output. The values discarded during pooling cannot therefore be recovered. The indices of the maxima only provide information about the locations of the retained values.*

Remark 31. *Unpooling should also be distinguished from a transposed convolution. Both operations can increase the spatial resolution of a feature map, but a transposed convolution contains learnable parameters, whereas max-unpooling uses the indices recorded during max pooling and does not learn additional parameters. Upsampling operations are used in architectures whose output has the same spatial resolution as the input, such as convolutional autoencoders or segmentation networks.*

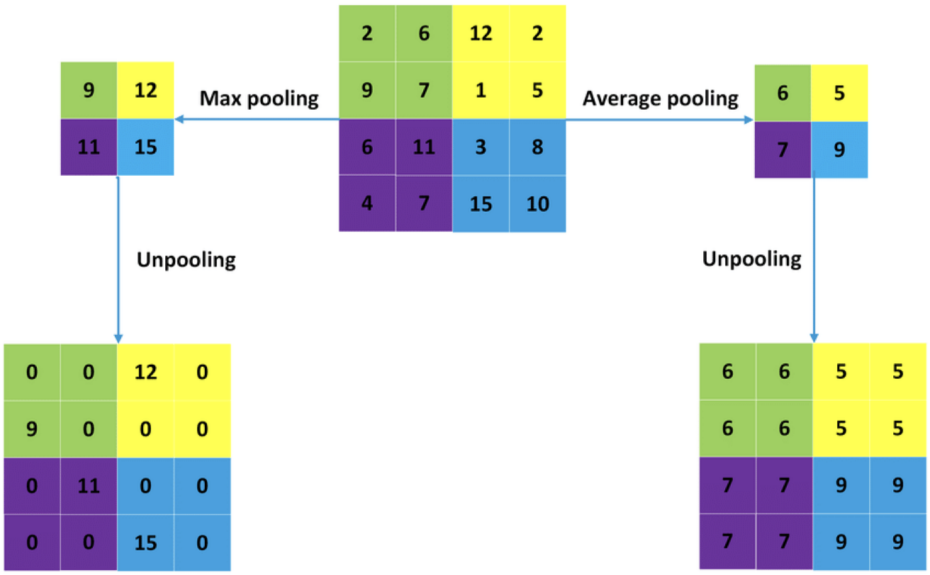


Figure 7: Example of max and average pooling layers with dimension 2×2 and stride 2 on a matrix of dimension 4×4, and of the corresponding max and average unpooling layers applied on the reduced matrices

5.4 Definition of a CNN

After introducing the convolutional layer and the pooling operation, we can now give a mathematical description of a convolutional neural network.

Definition 4. *Let*

$$X_0 \in \mathbb{R}^{H_0 \times W_0 \times C_0}$$

denote an input tensor. A Convolutional Neural Network (CNN) is a neural network in which the main feature-extraction part is composed of layers operating on spatially structured tensors, typically using convolutional layers, pointwise nonlinearities, pooling operations, and possibly normalization or skip connections.

More generally, a CNN can be represented as a composition

$$F \circ f_L \circ \cdots \circ f_2 \circ f_1,$$

where each intermediate mapping

$$f_i : \mathbb{R}^{H_{i-1} \times W_{i-1} \times C_{i-1}} \longrightarrow \mathbb{R}^{H_i \times W_i \times C_i}$$

operates on a spatial tensor. In particular, the mappings f_i may represent convolutional layers combined with activation functions, pooling operations, normalization layers, or other architectural components.

The final mapping

$$F : \mathbb{R}^{H_L \times W_L \times C_L} \longrightarrow \mathbb{R}^p$$

maps the resulting feature representation to the output space of the task. Depending on the application, F may contain fully connected layers, a global pooling operation, or other transformations.

Remark 32. *The defining characteristic of a convolutional neural network is not simply the presence of convolutional layers, but the use of operations that exploit the spatial structure of the input. In particular, convolutional layers combine local connectivity with weight sharing: the same learnable kernel is applied at different spatial locations.*

5.5 Receptive field

An important consequence of the local structure of convolutional layers is that each output feature depends only on a restricted portion of the original input. This portion of the input is called the *receptive field* of the feature.

More precisely, consider a neuron located at position (i, j) in a feature map of a convolutional neural network. Its receptive field is the set of spatial positions in the original input that can influence the value of this neuron through the successive operations of the network.

For a single convolutional layer with a kernel of spatial size $k_h \times k_w$, the receptive field of an output neuron is precisely the region of the input covered by the kernel. In the case of a square kernel of size $k \times k$, the receptive field is therefore $k \times k$.

As convolutional layers are stacked, the receptive field of a neuron in a deeper layer generally becomes larger. Indeed, a neuron in the second layer depends on several neurons of the first layer, each of which already depends on a region of the original input. Consequently, successive convolutional layers progressively combine information from increasingly larger regions of the input.

This observation provides a mathematical way of understanding how CNNs progressively combine local information into increasingly global representations.

5.5.1 Receptive field and effective jump

To derive the receptive field of a deep convolutional network, it is useful to introduce the notion of *effective jump*.

Consider a sequence of L convolutional layers. For each layer l, let k_l denote the spatial size of the kernel and s_l its stride. For simplicity, we consider square kernels and equal horizontal and vertical strides. The same reasoning can be applied independently to the two spatial dimensions when the kernels or strides are different.

Let R_l denote the size of the receptive field of a neuron at layer l, measured with respect to the original input. We initialize

$$R_0 = 1,$$

since a neuron in the input layer depends only on itself.

We also introduce the *effective jump* J_l, which represents the distance, measured in the original input, between the centers of the receptive fields of two neighboring neurons in layer l.

At the input layer, two neighboring neurons are separated by one pixel. Hence,

$$J_0 = 1.$$

Suppose now that layer l has kernel size k_l and stride s_l. Two neighboring neurons in this layer are separated by s_l positions in the previous layer. Therefore, their corresponding receptive fields in the original input are separated by

$$J_l = s_l J_{l-1}.$$

Furthermore, the kernel of layer l covers k_l consecutive positions in the previous layer. The distance between the first and the last of these positions is therefore

$$(k_l - 1)J_{l-1}$$

when measured in the coordinates of the original input. Consequently, the receptive field increases by this quantity, giving

$$R_l = R_{l-1} + (k_l - 1)J_{l-1}.$$

We therefore obtain the following recurrence relations:

$$\boxed{\begin{aligned} R_l &= R_{l-1} + (k_l - 1)J_{l-1}, \\ J_l &= s_l J_{l-1}, \end{aligned}} \tag{4}$$

with initial conditions

$$R_0 = 1, \qquad J_0 = 1.$$

Proposition 3. *Consider a sequence of L convolutional layers with kernel sizes $k_1, \dots, k_L$ and strides $s_1, \dots, s_L$. The receptive field size of a neuron in layer L, measured with respect to the original input, is*

$$\boxed{R_L = 1 + \sum_{l=1}^{L} (k_l - 1) \prod_{m=1}^{l-1} s_m}. \tag{5}$$

Exercise 10. *Prove Proposition 3.*

5.5.2 The case of unit stride

A particularly important special case occurs when all convolutional layers have stride one:

$$s_1 = s_2 = \cdots = s_L = 1.$$

In this case,

$$J_l = 1 \qquad \text{for all } l,$$

and Equation (5) simplifies to

$$\boxed{R_L = 1 + \sum_{l=1}^{L} (k_l - 1).} \tag{6}$$

For example, consider a network composed of three successive convolutional layers, each using a 3×3 kernel and a stride of one. The receptive field of a neuron in the third layer is

$$R_3 = 1 + (3-1) + (3-1) + (3-1) = 7.$$

Thus, despite using only 3×3 kernels, a neuron in the third layer can depend on a 7×7 region of the original input.

This illustrates an important principle of convolutional neural networks: *stacking several small kernels allows the network to progressively increase its receptive field while introducing nonlinear transformations between the successive convolutional operations.*

Remark 33. *The receptive field describes the region of the input that can potentially influence a neuron. It does not necessarily mean that every pixel within this region has an equally important influence on the output. In particular, the actual contribution of different input pixels depends on the learned weights and the nonlinearities of the network.*

Exercise 11. *Consider two convolutional architectures, both using stride* 1 *and no padding.*

1. *Consider a single convolutional layer with a* 7×7 *kernel. Compute the receptive field of a neuron in its output feature map with respect to the input image.*
2. *Consider three successive convolutional layers, each using a* 3×3 *kernel. Compute the receptive field of a neuron in the final feature map with respect to the input image.*
3. *Compare the receptive fields obtained in Questions 1 and 2. What do you observe?*
4. *Assume that the input has* C_{in} *channels and that each convolutional layer produces* C_{out} *feature maps. Compute the number of learnable parameters in:*
   (a) *the single* 7×7 *convolutional layer;*
   (b) *the three successive* 3×3 *convolutional layers.*

   Ignore the bias terms for simplicity.
5. *Compare the two architectures in terms of:*
   - *their receptive field;*
   - *their number of learnable parameters;*
   - *the number of nonlinear activation functions that can be applied between the convolutional layers.*

   Based on these observations, explain why stacking several small convolutional kernels can be preferable to using a single large kernel.

5.6 Brief historical evolution of convolutional networks

In practice, a CNN for classification (Definition 4) is usually organized in three parts:

- **Feature extraction**, generally a sequence of convolutional and pooling layers;
- **Fully connected layers**, i.e. a multilayer perceptron applied to the extracted features;

- **Output layer**, which returns the prediction, generally a probability distribution over the classes. This layer strongly depends on the task.

Remark 34. *An activation function is applied pointwise to the feature map returned by a convolutional layer. Early networks such as LeNet-5 used the hyperbolic tangent or the sigmoid function; since AlexNet, the ReLU function and its variants have become the standard choice, and recent architectures also use smooth variants such as GELU or SiLU.*

There is no unique way to build a CNN: the design depends on what matters most for the application. Here are examples of criteria to take into account:

- the accuracy;
- the size of the network, which matters if it is deployed on a mobile phone;
- the inference time on a CPU or a GPU.

In the following sections, we present a non-exhaustive list of important CNNs in chronological order.

5.6.1 LeNet-5

LeNet-5 was proposed by LeCun *et al.* in 1998 [12]. It is a convolutional neural network trained with the backpropagation algorithm for handwritten character recognition.

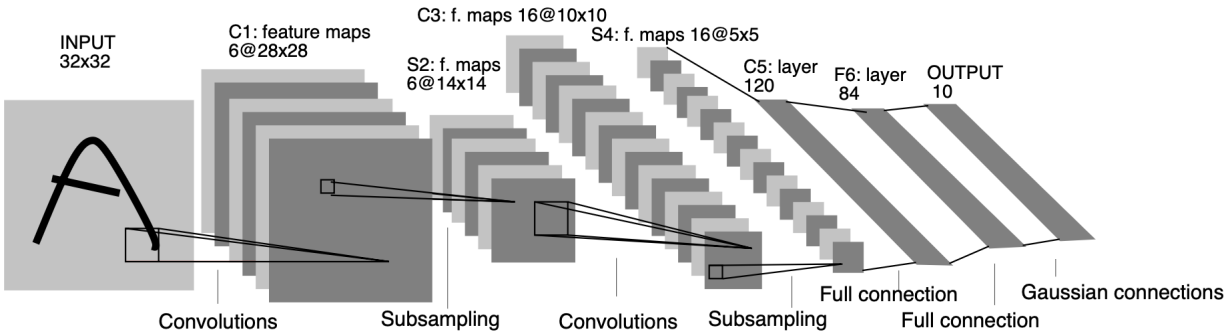


Figure 8: Architecture of LeNet-5 [12].

Layer	Type	Maps	Size	Kernel size	Stride	Activation
Out	Fully connected (RBF)	-	10	-	-	RBF
F6	Fully connected MLP	-	84	-	-	tanh
C5	Convolution	120	1 x 1	5 x 5	1	tanh
S4	Average pooling	16	5 x 5	2 x 2	2	tanh
C3	Convolution	16	10 x 10	5 x 5	1	tanh
S2	Average pooling	6	14 x 14	2 x 2	2	tanh
C1	Convolution	6	28 x 28	5 x 5	1	tanh
In	Input	1	32 x 32	-	-	-

Table 1: Details on the LeNet-5 architecture.

Layer $F6$ contains 84 units and returns a vector $x \in \mathbb{R}^{84}$ where:

$$\forall i \in [\![1, 84]\!] , \quad x_i = f(a_i) ,$$

a_i being the dot product between the input vector of $F6$ (the 120 outputs of $C5$) and the weight vector of unit i, plus a bias. The function f is a scaled hyperbolic tangent defined by:

$$\forall a \in \mathbb{R} , \quad f(a) = A \tanh(S\, a)$$

where $A = 1.7159$ and $S = 2/3$ in the original paper. The output of the final layer is a vector $y \in \mathbb{R}^{10}$ given by:

$$\forall i \in [\![1, 10]\!], \quad y_i = \sum_{j=1}^{84} (x_j - w_{ij})^2.$$

In the original paper, the output layer is composed of Euclidean Radial Basis Function (RBF) units, one for each class, each with 84 inputs. In other words, each output RBF unit computes the squared Euclidean distance between the input vector x and its parameter vector $w_i \in \mathbb{R}^{84}$, and the predicted class is the one with the smallest output. Training essentially minimizes the average, over the training set, of the output y_i associated with the correct class, i.e. a mean squared error between x and the prototype of the correct class (the original paper adds a discriminative term to this criterion).

Remark 35. *In modern implementations, the RBF output layer is replaced by a fully connected layer with* 10 *outputs followed by a softmax, and the network is trained with the cross-entropy loss.*

5.6.2 AlexNet

AlexNet is a convolutional neural network that won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) in 2012 [11]. It was developed by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton. Unlike LeNet-5, which was designed to classify handwritten digits, AlexNet was built to classify full-color images into 1000 classes.

Its architecture shares some similarities with LeNet-5 but introduces several important innovations:

- **ReLU activation:** a ReLU (Rectified Linear Unit) activation function is applied after each convolution and fully connected layer, replacing the hyperbolic tangent used in earlier networks.
- **Deeper network:** AlexNet stacks five convolutional layers, three of them consecutively, enabling it to learn more complex features.
- **Overlapping pooling:** max pooling uses 3×3 windows with stride 2, so that neighboring windows overlap.
- **Regularization techniques:**
  - **Dropout:** a dropout rate of 50% is applied in the fully connected layers F9 and F10 to prevent overfitting.
  - **Data augmentation:** the training images are augmented with random 224×224 crops of 256×256 images, horizontal reflections, and random perturbations of the color intensities, which improves generalization.
- **Local Response Normalization (LRN):** this technique is applied after the ReLU activations of layers C1 and C3, helping to improve generalization.

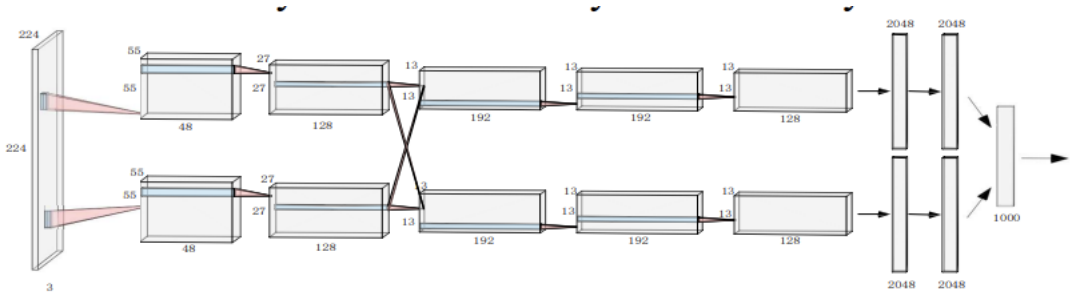


Figure 9: Architecture of AlexNet [11]. The network was split across two GPUs, which explains the two parallel streams.

Definition 5. *Let the kernels of a layer be indexed by $i \in \{0, \dots, K-1\}$. The LRN function applied to the outputs of layers C1 and C3 is defined by*

$$b^i_{u,v} = a^i_{u,v} \left(k + \alpha \sum_{j=j_{\text{low}}}^{j_{\text{up}}} (a^j_{u,v})^2 \right)^{-\beta},$$

with $j_{\text{low}} = \max\big(0, i - \lfloor r/2 \rfloor\big)$ and $j_{\text{up}} = \min\big(K-1, i + \lfloor r/2 \rfloor\big)$. Here we have:

Layer	Type	Maps	Size	Kernel size	Stride	Padding	Activation
Out	Fully connected MLP	-	1000	-	-	-	Softmax
F10	Fully connected MLP	-	4096	-	-	-	ReLU
F9	Fully connected MLP	-	4096	-	-	-	ReLU
S8	Max pooling	256	6 x 6	3 x 3	2	valid	-
C7	Convolution	256	13 x 13	3 x 3	1	same	ReLU
C6	Convolution	384	13 x 13	3 x 3	1	same	ReLU
C5	Convolution	384	13 x 13	3 x 3	1	same	ReLU
S4	Max pooling	256	13 x 13	3 x 3	2	valid	-
C3	Convolution	256	27 x 27	5 x 5	1	same	ReLU
S2	Max pooling	96	27 x 27	3 x 3	2	valid	-
C1	Convolution	96	55 x 55	11 x 11	4	valid	ReLU
In	Input	3 (RGB)	227 x 227	-	-	-	-

Table 2: Details on the AlexNet architecture. The spatial sizes can be checked with the formula of Exercise 9.

- $a^i_{u,v}$ *is the activity of the neuron computed by applying kernel* i *at position* (u, v)*, after the ReLU;*
- $b^i_{u,v}$ *is the normalized output;*
- K *is the total number of kernels in the layer, and* r *is the number of adjacent kernels involved in the sum.*

Remark 36. *This sort of response normalization implements a form of lateral inhibition inspired by the type found in real neurons, creating competition for big activities among the neuron outputs computed using different kernels.*

In the original paper, the hyperparameters taken for LRN are $r = 5$*,* $\alpha = 10^{-4}$*,* $\beta = 0.75$ *and* $k = 2$*. LRN has since been abandoned in favor of Batch Normalization.*

Training conditions:

- *Loss function:* cross-entropy;
- *Optimizer:* stochastic gradient descent with a batch size of 128 examples, momentum of 0.9, and weight decay of 0.0005. The learning rate was initialized at 0.01 and divided by 10 each time the validation error stopped improving.

Remark 37. *More precisely, the update rule for a weight* w *is:*

$$v_{i+1} = 0.9\, v_i - 0.0005\, \epsilon\, w_i - \epsilon\, \text{Average}_{D_i}\Big(\frac{\partial L}{\partial w}_{|w_i}\Big)$$

$$w_{i+1} = w_i + v_{i+1}$$

where i *is the iteration index,* v *is the momentum variable,* ϵ *is the learning rate, and* $\text{Average}_{D_i}\Big(\frac{\partial L}{\partial w}_{|w_i}\Big)$ *is the average over the* i*-th batch* D_i *of the derivative of the loss function with respect to* w*, evaluated at* w_i*. The term* $-0.0005\, \epsilon\, w_i$ *is* $-\epsilon$ *times the gradient of* $\frac{0.0005}{2}\, w^2$*: weight decay is exactly the* ℓ_2 *regularization of Section 4.1, combined with the momentum method of Section 3.*

5.6.3 VGGNets

VGGNets are a series of convolutional neural networks proposed by the Visual Geometry Group (VGG) of Oxford University [22], including VGG-11, VGG-11-LRN, VGG-13, VGG-16, and VGG-19. VGGNets secured the first place in the localization track and the second place in the classification track of the ImageNet Challenge 2014. The authors showed empirically that increasing the depth of the network, while keeping very small convolution kernels, improves performance up to a certain point.

Table 3 presents the architecture of the smallest VGG network. Compared with AlexNet, VGG-11 uses only 3×3 convolutions with stride 1 and "same" padding, and 2×2 max pooling with stride 2. The network is deeper (eight convolutional layers instead of five), and the spatial resolution is halved after each pooling layer while the number of channels is doubled, up to 512.

Layer	Type	Maps	Size	Kernel size	Stride	Padding	Activation
Out	Fully connected MLP	-	1000	-	-	-	Softmax
F15	Fully connected MLP	-	4096	-	-	-	ReLU
F14	Fully connected MLP	-	4096	-	-	-	ReLU
S13	Max pooling	512	7 x 7	2 x 2	2	valid	-
C12	Convolution	512	14 x 14	3 x 3	1	same	ReLU
C11	Convolution	512	14 x 14	3 x 3	1	same	ReLU
S10	Max pooling	512	14 x 14	2 x 2	2	valid	-
C9	Convolution	512	28 x 28	3 x 3	1	same	ReLU
C8	Convolution	512	28 x 28	3 x 3	1	same	ReLU
S7	Max pooling	256	28 x 28	2 x 2	2	valid	-
C6	Convolution	256	56 x 56	3 x 3	1	same	ReLU
C5	Convolution	256	56 x 56	3 x 3	1	same	ReLU
S4	Max pooling	128	56 x 56	2 x 2	2	valid	-
C3	Convolution	128	112 x 112	3 x 3	1	same	ReLU
S2	Max pooling	64	112 x 112	2 x 2	2	valid	-
C1	Convolution	64	224 x 224	3 x 3	1	same	ReLU
In	Input	3 (RGB)	224 x 224	-	-	-	-

Table 3: Details on the VGG-11 architecture.

Remark 38. *As shown in Exercise 11, a stack of three* 3×3 *convolutions has the same receptive field as a single* 7×7 *convolution, but uses* $27\,C^2$ *instead of* $49\,C^2$ *weights (for* C *input and output channels) and inserts three nonlinearities instead of one. This does not make VGG networks small: VGG-11 has about* 133 *million parameters, most of them in the fully connected layers.*

5.6.4 ResNet

Before the introduction of ResNet, research primarily focused on increasing the depth of convolutional neural networks to improve performance on benchmark tasks. However, this approach encountered several significant challenges:

- **Vanishing gradients:** gradients diminish rapidly towards zero, preventing the model from learning effectively during training.
- **Exploding gradients:** gradients can grow uncontrollably large, often due to error accumulation, leading to unstable training.
- **Degradation problem:** as the network depth increases, accuracy initially saturates and then begins to degrade, even though overfitting is not the cause (the training error itself increases).

Remark 39. *While the issues of vanishing and exploding gradients have largely been mitigated through techniques such as careful weight initialization and the use of normalization layers (e.g., Batch Normalization), enabling the training of moderately deep networks with stochastic gradient descent (SGD) and backpropagation, the degradation problem remains a major obstacle, especially in very deep architectures.*

ResNet (Residual Network), introduced by He *et al.* in 2015 [5], won the ILSVRC 2015 competition for image classification and detection. Its key innovation is the introduction of **residual connections**, designed to overcome the *degradation problem* observed in very deep networks such as VGG.

Formally, the core building block of a residual network is defined as:

$$y = \mathscr{F}(x, \{W_i\}) + x \ ,$$

where $\mathscr{F}(x, \{W_i\})$ represents the **residual mapping** to be learned, typically a stack of two or three convolutional layers. The shortcut (or skip) connection adds the input x directly to the output of $\mathscr{F}$, enabling efficient gradient propagation and stable training of very deep networks. When the dimensions of x and $\mathscr{F}(x, \{W_i\})$ differ, the shortcut is replaced by a linear projection, $y = \mathscr{F}(x, \{W_i\}) + W_s x$.

Remark 40. *The Jacobian of a residual block is*

$$\frac{\partial y}{\partial x} = I + \frac{\partial \mathscr{F}}{\partial x}.$$

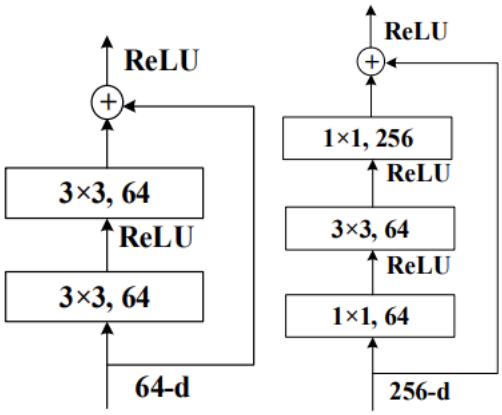


Figure 10: Structure of ResNet blocks. On the left, the structure of two-layer residual block. On the right, the structure of three-layer residual block

By the chain rule, the gradient backpropagated through a stack of residual blocks is a product of such matrices, so the identity term always provides a direct path for the gradient, even when $\partial\mathscr{F}/\partial x$ is small. Moreover, a block can represent the identity simply by setting $\mathscr{F} = 0$, so that adding blocks should not, in principle, increase the training error.

Figure 10 illustrates this residual connection structure used in the original ResNet architectures.

6 Transformer-based models

Transformer-based models are neural architectures designed to process sequences while modelling interactions between all relevant elements of the sequence. They were introduced in the encoder–decoder architecture of *Attention Is All You Need* [24] and have since become a central tool in natural language processing, computer vision, speech processing, and multimodal learning. Their main building block is *self-attention*, which constructs a representation of each token by aggregating information from other tokens in the sequence.

This section develops the main components of a Transformer with a column-vector convention. Thus, for a sequence of length T, an input representation is a matrix

$$X = \begin{bmatrix} x^{(1)} & \cdots & x^{(T)} \end{bmatrix} \in \mathbb{R}^{d_{\text{model}} \times T},$$

where $x^{(i)} \in \mathbb{R}^{d_{\text{model}}}$ is the representation of the token at position i.

6.1 Tokenization and dense embeddings

Text cannot be directly processed by a neural network: it first needs to be converted into a sequence of discrete symbols. This step is called *tokenization*. A token can be a character, a word, a subword, or a byte. Modern language models usually rely on subword tokenizers, such as Byte Pair Encoding (BPE) or WordPiece. Subword tokenization provides a useful compromise: frequent words may be represented by one token, whereas rare words can be decomposed into several smaller units.

Definition 6. *Let $\mathcal{V}$ be a finite vocabulary. A tokenizer maps a text sequence to a sequence of token indices*

$$(t_1, \dots, t_T), \qquad t_i \in \{1, \dots, |\mathcal{V}|\}.$$

An embedding layer *is a trainable matrix*

$$E \in \mathbb{R}^{d_{\text{model}} \times |\mathcal{V}|},$$

whose t_i-th column is the initial dense representation of token t_i.

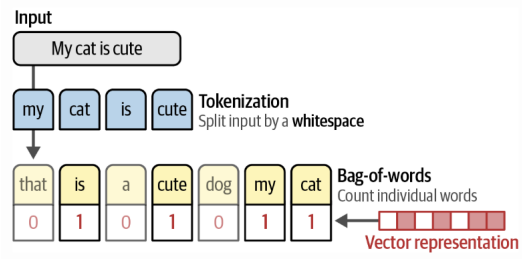


Figure 11: Sparse vector representation of the sentence "My cat is cute" using the vocabulary $\{\text{that}, \text{is}, \text{a}, \text{cute}, \text{dog}, \text{my}, \text{cat}\}$

Equivalently, if $e_{t_i} \in \mathbb{R}^{|\mathcal{V}|}$ denotes the one-hot vector associated with token t_i, its embedding can be written as

$$x^{(i)} = E e_{t_i} \in \mathbb{R}^{d_{\text{model}}}.$$

In practice, the implementation uses a lookup operation rather than explicitly multiplying by a one-hot vector.

From bag-of-words to contextual representations. The bag-of-words representation counts the occurrences of tokens in a text. It is simple and interpretable, but it discards word order and produces high-dimensional sparse vectors. Dense embeddings reduce this dimensionality and allow similarities between tokens to be learned from data. However, a classical embedding method such as word2vec assigns a single vector to each word type. Consequently, the two occurrences of *bank* in *river bank* and *bank loan* receive the same initial representation. Transformer layers produce *contextual embeddings*: the representation of a token is updated according to the other tokens appearing in the sequence.

Remark 41. *It is generally misleading to interpret one coordinate of an embedding as one independent semantic concept. Semantic information is distributed across the whole vector and, in a Transformer, across the sequence of successive layers.*

6.2 Scaled dot-product self-attention

Self-attention allows every token to select information from the tokens that are visible to it. Given an input matrix $X \in \mathbb{R}^{d_{\text{model}} \times T}$, define three trainable linear projections:

$$Q = W_Q X \in \mathbb{R}^{d_k \times T}, \tag{7}$$

$$K = W_K X \in \mathbb{R}^{d_k \times T}, \tag{8}$$

$$V = W_V X \in \mathbb{R}^{d_v \times T}, \tag{9}$$

where

$$W_Q, W_K \in \mathbb{R}^{d_k \times d_{\text{model}}}, \qquad W_V \in \mathbb{R}^{d_v \times d_{\text{model}}}.$$

The columns of Q, K, and V are respectively called *queries*, *keys*, and *values*. A query determines what information a position is looking for, a key determines how relevant a position is to that query, and a value is the information that may be transmitted.

The matrix of unnormalized attention scores is

$$S = \frac{Q^\top K}{\sqrt{d_k}} \in \mathbb{R}^{T \times T}.$$

The entry S_{ij} measures the compatibility between the query at position i and the key at position j. Applying softmax independently to every row of S gives the attention matrix

$$A = \text{softmax}_{\text{row}}(S), \qquad A_{ij} = \frac{\exp(S_{ij})}{\sum_{\ell=1}^{T} \exp(S_{i\ell})}.$$

The output of scaled dot-product attention is then

$$\text{Attention}(Q, K, V) = V \left[\text{softmax}_{\text{row}} \left(\frac{Q^\top K}{\sqrt{d_k}} \right) \right]^\top. \tag{10}$$

More explicitly, if $q^{(i)}$, $k^{(j)}$, and $v^{(j)}$ denote the corresponding column vectors, the new representation at position i is

$$\alpha_{ij} = \frac{\exp\left(\dfrac{(q^{(i)})^\top k^{(j)}}{\sqrt{d_k}} \right)}{\displaystyle\sum_{\ell=1}^{T} \exp\left(\frac{(q^{(i)})^\top k^{(\ell)}}{\sqrt{d_k}} \right)}, \tag{11}$$

$$z^{(i)} = \sum_{j=1}^{T} \alpha_{ij} v^{(j)}. \tag{12}$$

Thus, each $z^{(i)}$ is a weighted average of the visible value vectors.

Why divide by $\sqrt{d_k}$? Assume, as a simple model, that the coordinates of q and k are independent, centred random variables with variance 1. Then

$$\text{Var}(q^\top k) = d_k.$$

Without the scaling factor, the variance of the logits grows with the key dimension. Softmax may then become excessively concentrated, leading to small gradients for most entries. Dividing by $\sqrt{d_k}$ keeps the score variance approximately independent of d_k.

Remark 42. *Self-attention can process all positions in parallel during training, unlike a recurrent neural network. Nevertheless, the attention-score matrix has shape $T \times T$. Standard full attention therefore has quadratic dependence on the sequence length.*

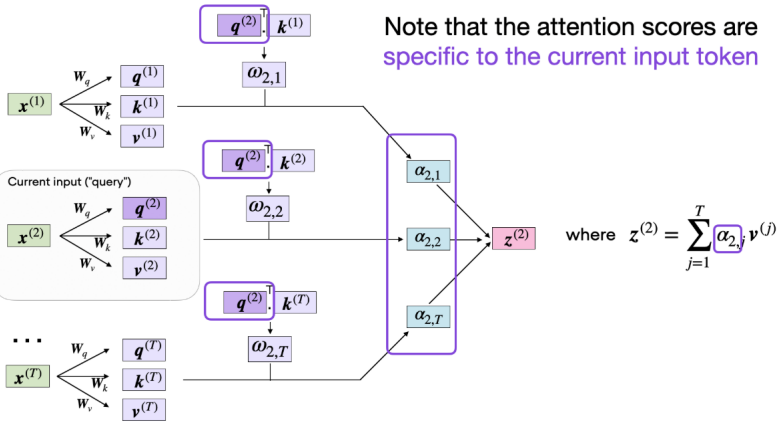


Figure 12: Attention scores calculated for a token represented by the dense vector $x^{(2)}$. The $\alpha_{2,j}$ are the attention scores of $x^{(2)}$, and $z^{(2)}$ is a dense vector which can be seen as a new vector representation of $x^{(2)}$. The integer T is the sequence length; the maximal length that a model can process is called its *context window*.

6.3 Attention masks and causal language modelling

Not every token should necessarily attend to every other token. An attention mask encodes which interactions are permitted. Let $M \in \mathbb{R}^{T\times T}$ be a mask whose entries are 0 for permitted interactions and $-\infty$ for forbidden interactions. Masked attention is defined by

$$\text{MaskedAttention}(Q,K,V;M) = V\left[\text{softmax}_{\text{row}}\left(\frac{Q^\top K}{\sqrt{d_k}} + M\right)\right]^\top. \tag{13}$$

In numerical implementations, $-\infty$ is replaced by a sufficiently large negative number.

For an autoregressive, or *causal*, language model, position i must not use tokens located in the future. The causal mask is

$$M_{ij} = \begin{cases} 0, & \text{if } j \leqslant i, \\ -\infty, & \text{if } j > i. \end{cases}$$

Consequently, the representation at position i depends only on tokens $1,\dots,i$. This makes it possible to model the joint distribution of a token sequence by the factorization

$$p_\theta(t_1,\dots,t_T) = \prod_{i=1}^{T} p_\theta(t_i \mid t_1,\dots,t_{i-1}). \tag{14}$$

Two other masks are frequently used:

- A *padding mask* prevents a model from attending to artificial padding tokens added to make sequences in a batch have the same length.
- An *encoder–decoder mask* is used in sequence-to-sequence models: decoder self-attention is causal, whereas cross-attention can attend to the encoder representations.

6.4 Multi-head attention

A single attention mechanism uses one similarity function and one collection of value vectors. Multi-head attention instead learns several projections in parallel. For head $r \in \{1,\dots,h\}$, let

$$H_r = \text{Attention}\left(W_Q^{(r)}X, W_K^{(r)}X, W_V^{(r)}X\right).$$

The outputs of all heads are concatenated vertically and projected back to the model dimension:

$$\text{MHA}(X) = W_O \begin{bmatrix} H_1 \\ \vdots \\ H_h \end{bmatrix}. \tag{15}$$

Usually,

$$d_k = d_v = \frac{d_{\text{model}}}{h}, \qquad W_O \in \mathbb{R}^{d_{\text{model}} \times h d_v}.$$

The different heads may learn different interaction patterns, for example short-range syntactic relations or longer-range semantic relations. Such interpretations should be treated cautiously: a head need not have one unique or human-interpretable role.

6.5 Why positional information is necessary

Without positional information, self-attention is permutation equivariant: if the input tokens are reordered, the output vectors are reordered in exactly the same way. Thus, the model has no intrinsic way to distinguish *the dog bit the man* from *the man bit the dog*.

Remark 43. *Let $P \in \mathbb{R}^{T \times T}$ be a permutation matrix. In the absence of positional information,*

$$\text{Attention}(W_Q XP, W_K XP, W_V XP) = \text{Attention}(W_Q X, W_K X, W_V X)P.$$

Indeed, the score matrix becomes $P^\top SP$, and row-wise softmax satisfies

$$\text{softmax}_{\text{row}}(P^\top SP) = P^\top \text{softmax}_{\text{row}}(S)P.$$

6.5.1 Sinusoidal positional encoding

The original Transformer adds a deterministic vector to each token embedding. Assume that d_{model} is even. For a position $i \in \{0, \dots, T-1\}$ and a frequency index $m \in \{0, \dots, d_{\text{model}}/2 - 1\}$, define

$$\text{PE}_{i,2m} = \sin\left(\frac{i}{10000^{2m/d_{\text{model}}}}\right), \tag{16}$$

$$\text{PE}_{i,2m+1} = \cos\left(\frac{i}{10000^{2m/d_{\text{model}}}}\right). \tag{17}$$

Writing

$$\text{PE} = \begin{bmatrix} \text{PE}_0 & \cdots & \text{PE}_{T-1} \end{bmatrix} \in \mathbb{R}^{d_{\text{model}} \times T},$$

the input to the first Transformer block is

$$X^{(0)} = \begin{bmatrix} Ee_{t_1} & \cdots & Ee_{t_T} \end{bmatrix} + \text{PE}.$$

Different coordinate pairs oscillate at different frequencies. By the trigonometric addition formulas, the encoding at position $i + r$ can be expressed as a linear transformation of the encoding at i, for every fixed offset r. This gives the model access to relative displacements as well as absolute positions.

Learned absolute positional embeddings replace the fixed sinusoidal vectors by trainable vectors. They are simple and effective, but are tied to the range of positions observed during training.

```python
import numpy as np
import matplotlib.pyplot as plt

def sinusoidal_positional_encoding(max_position, d_model):
    position = np.arange(max_position)[:, np.newaxis]
    # The original formula pos / 10000^(2m/d_model) is equivalent to pos * (1 / 10000^(2m/d_model)).
    # We use the version below for numerical stability
    div_term = np.exp(np.arange(0, d_model, 2) * -(np.log(10000.0) / d_model))

    pe = np.zeros((max_position, d_model))
    pe[:, 0::2] = np.sin(position * div_term)
    pe[:, 1::2] = np.cos(position * div_term)

    return pe

max_position = 100  # Maximum sequence length
d_model = 128        # Embedding dimension

pe = sinusoidal_positional_encoding(max_position, d_model)
```

```
plt.figure(figsize=(12, 8))
plt.imshow(pe, cmap='coolwarm', aspect='auto', vmin=-1, vmax=1)
plt.colorbar()
plt.title('Sinusoidal Positional Encoding')
plt.xlabel('Dimension')
plt.ylabel('Position')
plt.tight_layout()
plt.show()

plt.figure(figsize=(12, 6))
dimensions = [0, 21]
for d in dimensions:
    plt.plot(pe[:, d], label=f'Dim {d}')
plt.legend()
plt.title('Sinusoidal Positional Encoding for Specific Dimensions')
plt.xlabel('Position')
plt.ylabel('Value')
plt.tight_layout()
plt.show()
```

6.5.2 Rotary positional embeddings

Rotary positional embeddings (RoPE) encode position by rotating queries and keys rather than by adding a vector to the input. Consider the two-dimensional rotation matrix

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}.$$

For an even-dimensional query or key vector, RoPE applies a block-diagonal matrix

$$\mathscr{R}_i = \operatorname{diag}\left(R(i\omega_0), R(i\omega_1), \ldots, R\left(i\omega_{d_k/2-1}\right)\right),$$

where the frequencies ω_m decrease with m, typically according to a geometric schedule. Queries and keys at position i are replaced by

$$\widetilde{q}^{(i)} = \mathscr{R}_i q^{(i)}, \qquad \widetilde{k}^{(i)} = \mathscr{R}_i k^{(i)}.$$

The values are usually left unchanged.

The key property is

$$\big(R(i\theta)q\big)^\top \big(R(j\theta)k\big) = q^\top R((j-i)\theta)k.$$

Hence, the attention score between two rotated vectors depends on the relative position $j - i$. RoPE is widely used in contemporary decoder-only language models because it injects relative-position information directly into the query–key dot products.

6.6 A Transformer block

A Transformer consists of a stack of blocks. Each block combines multi-head attention, a position-wise feed-forward network, residual connections, and normalisation layers. A common modern arrangement is the *pre-normalised* form:

$$U^{(\ell)} = X^{(\ell-1)} + \mathrm{MHA}^{(\ell)}\left(\mathrm{LN}_1^{(\ell)}(X^{(\ell-1)})\right), \tag{18}$$

$$X^{(\ell)} = U^{(\ell)} + \mathrm{FFN}^{(\ell)}\left(\mathrm{LN}_2^{(\ell)}(U^{(\ell)})\right). \tag{19}$$

Dropout is often added during training, but is omitted from Equations (18)–(19) for clarity.

The original Transformer [24] instead used the *post-normalised* form, in which normalisation is applied after each residual addition:

$$U^{(\ell)} = \mathrm{LN}_1^{(\ell)}\left(X^{(\ell-1)} + \mathrm{MHA}^{(\ell)}(X^{(\ell-1)})\right), \qquad X^{(\ell)} = \mathrm{LN}_2^{(\ell)}\left(U^{(\ell)} + \mathrm{FFN}^{(\ell)}(U^{(\ell)})\right).$$

This is the arrangement shown by the "Add & Norm" boxes of Figure 13. The pre-normalised form keeps an identity path from the input to the output of the whole stack, which tends to make the training of deep Transformers more stable.

Layer normalisation is applied independently at each token position. For $x \in \mathbb{R}^{d_{\text{model}}}$, it has the form

$$\mathrm{LN}(x) = \gamma \odot \frac{x - \mu(x)\mathbf{1}}{\sqrt{\sigma^2(x) + \varepsilon}} + \beta,$$

where $\mu(x)$ and $\sigma^2(x)$ are respectively the mean and variance of the coordinates of x, γ and β are trainable vectors, and $\varepsilon > 0$ prevents division by zero.

The feed-forward network is applied independently and identically at every position. For one token vector u, a standard form is

$$\mathrm{FFN}(u) = W_2\, \sigma(W_1 u + b_1) + b_2,$$

where

$$W_1 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}, \qquad W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}},$$

and σ is a nonlinear activation function, such as GELU. The expansion dimension d_{ff} is often several times larger than d_{model}. In many recent models, this MLP is replaced by a gated variant such as SwiGLU.

Remark 44. *Residual connections give each layer access to the input of that layer and help gradients propagate through deep networks (see also the ResNet blocks of Section 5). Attention mixes information across positions, whereas the feed-forward network transforms the features at each position separately. The alternation of these two operations is a central design principle of the Transformer.*

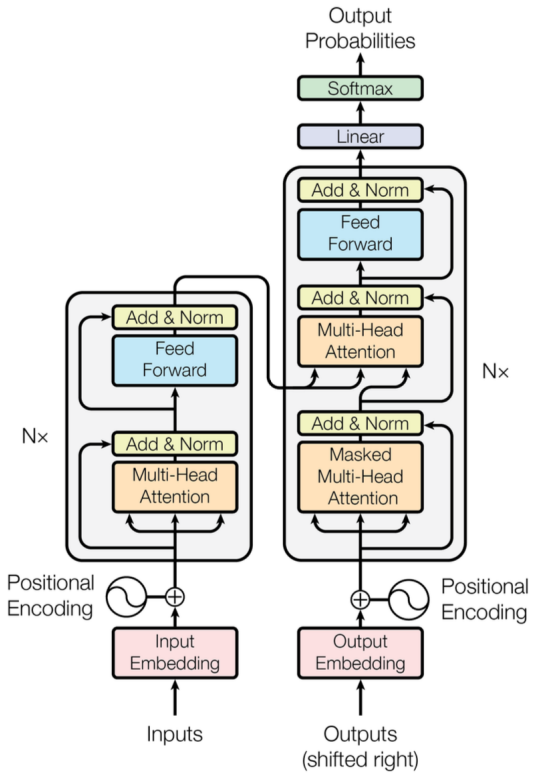


Figure 13: The original Transformer architecture [24]: an encoder (left) and a decoder (right), each made of N post-normalised blocks.

6.7 Encoder-only, decoder-only, and encoder–decoder models

The original Transformer is an encoder–decoder model, but the same building blocks can be organised in several ways.

- **Encoder-only models.** The encoder uses bidirectional self-attention: each non-padding token can attend to all tokens in the input. Such models are appropriate for representation, classification, retrieval, and token-labelling tasks. BERT is a classical example. Its pretraining objective reconstructs masked tokens from their context.

- **Decoder-only models.** A decoder-only Transformer uses causal self-attention and predicts the next token. GPT-type models belong to this family. They are well suited to text generation because the training objective coincides with left-to-right generation.
- **Encoder–decoder models.** The encoder builds bidirectional representations of a source sequence. The decoder then contains causal self-attention and *cross-attention*, where its queries come from the decoder and its keys and values come from the encoder. This architecture is natural for conditional generation tasks, such as translation, summarisation, and question answering. T5 and BART are representative examples.

If H_{enc} denotes the encoder output and H_{dec} a decoder representation, one cross-attention layer takes the form

$$\text{Attention}\left(W_Q H_{\text{dec}},\, W_K H_{\text{enc}},\, W_V H_{\text{enc}}\right).$$

It allows each generated token to retrieve relevant information from the source sequence.

6.8 Training objective and token generation

For a decoder-only language model, the final hidden representation $h^{(i)} \in \mathbb{R}^{d_{\text{model}}}$ is mapped to vocabulary logits:

$$s^{(i)} = W_{\text{vocab}} h^{(i)} + b_{\text{vocab}} \in \mathbb{R}^{|\mathscr{V}|}.$$

The probability of the next token is obtained by softmax:

$$p_\theta(t_{i+1} = a \mid t_{\leqslant i}) = \frac{\exp(s_a^{(i)})}{\sum_{b=1}^{|\mathscr{V}|} \exp(s_b^{(i)})}.$$

Given a training sequence $(t_1, \ldots, t_T)$, the autoregressive negative log-likelihood is

$$\mathscr{L}_{\text{AR}}(\theta) = -\sum_{i=1}^{T-1} \log p_\theta(t_{i+1} \mid t_1, \ldots, t_i). \tag{20}$$

Minimising this loss is equivalent to maximising the likelihood of the training corpus under the model. Section 7.1 revisits this objective in more detail.

At inference time, the model repeatedly predicts a distribution for the next token, selects or samples a token, appends it to the current sequence, and continues until an end-of-sequence token or a length limit is reached. Common decoding strategies include greedy decoding, beam search, temperature scaling, top-k sampling, and nucleus sampling (see Section 7.5).

6.9 Computational considerations

For one layer with sequence length T and model dimension d_{model}, forming the attention matrix requires $O(T^2 d_{\text{model}})$ arithmetic operations and $O(T^2)$ memory if the matrix is explicitly stored. This quadratic term becomes a major limitation for long sequences.

During autoregressive generation, previously computed keys and values can be stored in a *key–value cache*. When a new token is generated, its query only needs to be compared with the cached keys, rather than recomputing all past projections. The cache substantially reduces repeated computation, although its memory consumption grows linearly with the generated sequence length and with the number of layers.

Many extensions aim to improve long-context efficiency. Examples include memory-efficient attention kernels, sparse attention patterns, compressed memories, and linear-attention approximations. These methods modify the computational trade-off, but the standard scaled dot-product attention defined in Equation (10) remains the conceptual foundation.

6.10 Exercises

1. **Dimensions and parameter count.** Let $d_{\text{model}} = 768$, $h = 12$, and $d_{\text{ff}} = 3072$. Determine the dimensions of the projection matrices in one multi-head attention layer. Compute the number of parameters in its four projection matrices, ignoring biases. Then compute the number of parameters in the two linear layers of the feed-forward network.

2. **Convex-combination property.** Prove that every row of the attention matrix A is a probability vector. Deduce that, for each position i, the output $z^{(i)}$ belongs to the convex hull of the value vectors $\{v^{(1)}, \dots, v^{(T)}\}$.

3. **The scaling factor.** Suppose that the coordinates of $q, k \in \mathbb{R}^{d_k}$ are independent, centred, and have variance 1. Show that
$$\mathbb{E}[q^\top k] = 0, \qquad \mathrm{Var}(q^\top k) = d_k.$$
Explain why this supports the scaling used in Equation (10).

4. **Permutation equivariance.** Let P be a $T \times T$ permutation matrix. Prove rigorously that self-attention without positional information is permutation equivariant:
$$\mathrm{Attention}\left(W_Q X P, W_K X P, W_V X P\right) = \mathrm{Attention}\left(W_Q X, W_K X, W_V X\right) P.$$
Why does this make positional information indispensable for language?

5. **Causal masking.** Prove that the output at position i of the masked attention mechanism with the causal mask depends only on $x^{(1)}, \dots, x^{(i)}$. Explain why this property is needed when training with the objective in Equation (20).

6. **Sinusoidal position shifts.** Fix a frequency ω. Using the angle-addition identities, show that
$$\begin{bmatrix} \sin((i+r)\omega) \\ \cos((i+r)\omega) \end{bmatrix} = R(-r\omega) \begin{bmatrix} \sin(i\omega) \\ \cos(i\omega) \end{bmatrix}.$$
Interpret the result in terms of relative positions.

7. **RoPE and relative position.** Prove the rotation identity used in the RoPE subsection. Extend it to the block-diagonal matrix $\mathscr{R}_i$ and explain precisely why the query–key dot product depends on $j - i$.

8. **Gradient of cross-entropy.** Let $p = \mathrm{softmax}(s)$ and let e_y be the one-hot vector of the correct class. For $\ell(s) = -\log p_y$, prove that
$$\nabla_s \ell = p - e_y.$$

9. **Complexity and key–value caching.** Derive the time complexity of full self-attention during training. Then compare the cost of generating a sequence of length T with and without a key–value cache. State clearly which quantities are stored in the cache.

10. **A small numerical example.** Consider three scalar tokens with $W_Q = W_K = W_V = [1]$ and no mask. Compute the attention matrix and output explicitly. Repeat the calculation with a causal mask. What changes at the first and second positions?

7 Large Language Models

A *large language model* (LLM) is a neural probabilistic model of discrete token sequences. In this course, we focus primarily on *decoder-only* Transformer models, which generate text autoregressively: each token is predicted from the tokens that precede it.

Notation. In this section, tokens are denoted by $w_1, \dots, w_T$ (rather than $t_1, \dots, t_T$ as in Section 6), positions are indexed by t, and $|\mathscr{V}|$ denotes the size of the vocabulary $\mathscr{V}$. Once a sequence has been tokenized, its embeddings are arranged according to the column-token convention adopted throughout this course,

$$X = \begin{bmatrix} x_1 & x_2 & \cdots & x_T \end{bmatrix} \in \mathbb{R}^{d_{\text{model}} \times T},$$

where x_t denotes the embedding of the token at position t.

More precisely, let $w_{1:T} = (w_1, \dots, w_T)$ be a sequence of tokens in a vocabulary $\mathscr{V}$. An LLM with parameters θ defines a probability distribution over complete sequences through the autoregressive factorization

$$p_\theta(w_{1:T}) = \prod_{t=1}^{T} p_\theta\big(w_t \mid w_{1:t-1}\big).$$

Thus, at each position, the model produces a categorical distribution over the next token, using only the preceding context. Repeatedly sampling or selecting from these conditional distributions yields a generated continuation.

The adjective *large* does not refer to a universal threshold on the number of parameters. Rather, it describes a training regime in which the number of parameters, the quantity and quality of training data, and the available computational budget are jointly scaled. This scaling can yield highly capable *base models*, able to continue text, perform in-context learning, and transfer to a broad range of tasks. These behaviours should nevertheless be understood as empirical properties of trained models, rather than as guarantees of correctness or reliable reasoning.

Pre-training alone does not turn a base model into a useful conversational assistant. A model trained solely to predict the next token may produce unhelpful, unsafe, outdated, or factually incorrect continuations. Modern LLM systems therefore usually include a post-training stage, in which the model is adapted to follow instructions and to better reflect human preferences. Moreover, the final response depends not only on the learned probability distribution, but also on the decoding strategy used at inference time.

Finally, the same principle can be extended beyond text. Images, audio signals, or other modalities can first be transformed into sequences of continuous representations, which then provide additional context to a language model. This idea forms the basis of multimodal language models (Section 8).

The following subsections examine successively the autoregressive training objective, pre-training, the role of scaling, post-training and alignment methods, and decoding strategies. We conclude with an overview of the capabilities and limitations of LLMs.

7.1 Autoregressive language modelling and tokenization

A language model does not directly manipulate raw text. Instead, a *tokenizer* converts a string into a finite sequence of discrete symbols called *tokens*. Formally, if $\mathscr{S}$ denotes the set of possible input strings and $\mathscr{V}$ is a finite vocabulary, a tokenizer is a map

$$\tau : \mathscr{S} \longrightarrow \mathscr{V}^*, \qquad s \longmapsto (w_1, \dots, w_T),$$

where $\mathscr{V}^*$ denotes the set of all finite sequences with elements in $\mathscr{V}$.

A token is not necessarily a word. Modern LLMs typically use subword or byte-level tokenizers: a frequent word may be represented by a single token, whereas a rare word, a punctuation mark, or part of a word may be represented by several tokens. For example, byte-pair encoding tokenizers start from a basic alphabet of characters or bytes and iteratively merge frequent adjacent symbols. Such a construction makes it possible to represent arbitrary text with a finite vocabulary while retaining frequently occurring patterns as short token sequences.

Special tokens are usually added to the vocabulary in order to mark structural elements of a sequence. In particular, BOS and EOS denote respectively the beginning and the end of a sequence. The latter gives the model a mechanism for deciding when generation should stop.

Each token $w_t \in \mathscr{V}$ is first represented by its one-hot vector $\mathbf{e}(w_t) \in \mathbb{R}^{|\mathscr{V}|}$. A learned embedding matrix

$$E \in \mathbb{R}^{d_{\text{model}} \times |\mathscr{V}|}$$

then maps this discrete representation to a dense vector,

$$\bar{x}_t = E\,\mathbf{e}(w_t) \in \mathbb{R}^{d_{\text{model}}}.$$

Thus, the same token is initially assigned the same embedding wherever it appears. Its representation becomes context-dependent only after it has been processed by the successive Transformer blocks.

The model must also receive positional information. For an architecture based on additive positional embeddings, one writes

$$x_t = \bar{x}_t + p_t,$$

where $p_t \in \mathbb{R}^{d_{\text{model}}}$ is the positional embedding at position t. In the case of RoPE, positional information is instead introduced by position-dependent rotations of the queries and keys inside the attention mechanism; in that case, no additive vector p_t is required at the input. With the column-token convention, the input sequence is represented by

$$X^{(0)} = \begin{bmatrix} x_1 & x_2 & \cdots & x_T \end{bmatrix} \in \mathbb{R}^{d_{\text{model}} \times T}.$$

The central modelling assumption of a decoder-only LLM is the *autoregressive factorization* of the probability of a token sequence:

$$p_\theta\big(w_{1:T} \mid \text{BOS}\big) = \prod_{t=1}^{T} p_\theta\big(w_t \mid \text{BOS}, w_{1:t-1}\big),$$

where θ denotes all trainable parameters of the model. In other words, the probability of the next token may depend on the previous tokens, but not on tokens that occur later in the sequence.

This constraint is enforced by the causal attention mask of Section 6.3: after applying the softmax, all attention weights assigned to future positions are equal to zero, so that the representation at position i depends only on the tokens at positions $1, \ldots, i$.

During training, the input and target sequences are shifted by one position. For a sequence $w_{1:T}$, the model receives

$$\widetilde{w}_{1:T} = \big(\text{BOS}, w_1, \ldots, w_{T-1}\big)$$

and is trained to predict the targets

$$\big(w_1, \ldots, w_T\big).$$

All target positions can therefore be evaluated in parallel during training, even though the model remains autoregressive.

Let

$$H^{(L)} = \begin{bmatrix} h_1^{(L)} & h_2^{(L)} & \cdots & h_T^{(L)} \end{bmatrix} \in \mathbb{R}^{d_{\text{model}} \times T}$$

denote the output of the L-th (last) decoder block when the shifted sequence is given as input (Figure 14). At each position t, a linear output layer produces the logits

$$\ell_t = W_{\text{out}} h_t^{(L)} + b_{\text{out}} \in \mathbb{R}^{|\mathscr{V}|},$$

where

$$W_{\text{out}} \in \mathbb{R}^{|\mathscr{V}| \times d_{\text{model}}}, \qquad b_{\text{out}} \in \mathbb{R}^{|\mathscr{V}|}.$$

The corresponding next-token probability distribution is obtained through the softmax function:

$$\pi_t = \text{softmax}(\ell_t), \qquad \pi_{t,v} = \frac{\exp(\ell_{t,v})}{\sum_{v' \in \mathscr{V}} \exp(\ell_{t,v'})}.$$

Hence,

$$\pi_{t,v} = p_\theta\big(w_t = v \mid \text{BOS}, w_{1:t-1}\big).$$

Given a training corpus

$$\mathscr{D} = \Big\{ w_{1:T_n}^{(n)} \Big\}_{n=1}^{m},$$

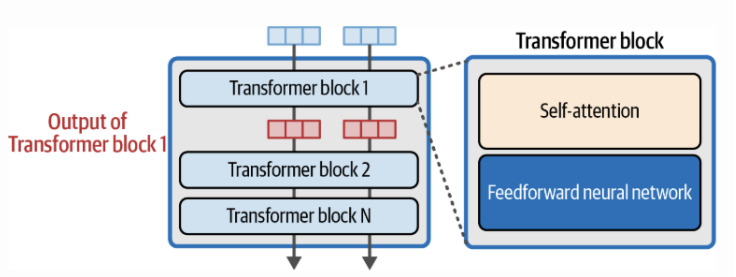


Figure 14: A decoder-only LLM is a stack of Transformer blocks: the output of each block is the input of the next one, and each block combines self-attention and a feed-forward network.

the model is trained by minimizing the average negative log-likelihood of the observed tokens. Writing

$$N_{\text{tok}} = \sum_{n=1}^{m} T_n,$$

the next-token prediction loss is

$$\begin{aligned}\mathscr{L}_{\text{NTP}}(\theta) &= -\frac{1}{N_{\text{tok}}} \sum_{n=1}^{m} \sum_{t=1}^{T_n} \log p_\theta\big(w_t^{(n)} \mid \text{BOS}, w_{1:t-1}^{(n)}\big) \\ &= -\frac{1}{N_{\text{tok}}} \sum_{n=1}^{m} \sum_{t=1}^{T_n} \mathbf{e}\big(w_t^{(n)}\big)^\top \log\big(\pi_t^{(n)}\big).\end{aligned}$$

This is the per-token average of the loss (20) of Section 6.8 over the corpus. The second expression is the usual multi-class cross-entropy loss. Minimizing this quantity is equivalent to maximum-likelihood estimation of the parameters θ. The training signal is called *self-supervised*: the target at each position is obtained directly from the input text, without requiring a separate human annotation.

A common summary of next-token prediction performance is the *perplexity*,

$$\text{PPL} = \exp\big(\mathscr{L}_{\text{NTP}}\big),$$

when the loss is expressed using natural logarithms. Perplexity should be compared with care: it is measured per token and therefore depends on the chosen tokenizer.

At inference time, the procedure is sequential. Starting from a prompt, the model predicts a distribution for the next token, selects or samples one token, appends it to the context, and repeats the process until an end-of-sequence token or another stopping criterion is reached. The choice of this selection rule, such as greedy decoding, temperature sampling, or nucleus sampling, is studied in Section 7.5.

7.2 Pre-training objective

The autoregressive factorization introduced above specifies how a decoder-only Transformer assigns a probability to a single token sequence. *Pre-training* consists in optimizing this model on a large collection of text sequences so that its predicted distribution matches, as closely as possible, the distribution of the training data.

At a conceptual level, let q denote the effective distribution over token sequences induced by the pre-training corpus, its preprocessing pipeline, and its sampling strategy. This distribution is generally a mixture of several data sources:

$$q = \sum_{k=1}^{K} \lambda_k q_k, \qquad \lambda_k \geqslant 0, \qquad \sum_{k=1}^{K} \lambda_k = 1,$$

where q_k denotes the distribution associated with the k-th source and λ_k its sampling weight. Thus, the behaviour of a pre-trained model depends not only on the total amount of data, but also on the relative proportions of the different sources.

Each document is tokenized and usually terminated by an EOS token. If $W_{1:T}$ denotes a random token sequence drawn from q, the population pre-training objective is the expected negative log-likelihood

$$\begin{aligned}\mathscr{R}_{\mathrm{pre}}(\theta) &= \mathbb{E}_{W_{1:T}\sim q}\left[-\log p_\theta\big(W_{1:T} \mid \mathrm{BOS}\big)\right] \\ &= \mathbb{E}_{W_{1:T}\sim q}\left[-\sum_{t=1}^{T}\log p_\theta\big(W_t \mid \mathrm{BOS}, W_{1:t-1}\big)\right].\end{aligned}$$

This is the cross-entropy between the data distribution and the model distribution:

$$\mathscr{R}_{\mathrm{pre}}(\theta) = H\big(q, p_\theta\big) = H(q) + D_{\mathrm{KL}}\big(q \,\|\, p_\theta\big),$$

where $H(q)$ is the entropy of the data distribution and $D_{\mathrm{KL}}(q\|p_\theta)$ is the Kullback–Leibler divergence from q to p_θ. Since $H(q)$ does not depend on θ, minimizing the pre-training loss is equivalent to minimizing

$$D_{\mathrm{KL}}\big(q \,\|\, p_\theta\big).$$

In the idealized case where the model family is sufficiently expressive and the data distribution is known exactly, the optimum is attained when $p_\theta = q$ (Exercise 1 of Section 7.7).

In practice, the distribution q is unknown and only a finite corpus is available. Pre-training therefore minimizes an empirical estimate of the population risk. Consider a mini-batch of B token sequences, padded to a common length $T_{\max}$. Let

$$\mu_{b,t} \in \{0, 1\}$$

denote a loss mask, equal to 1 when the target token at position t in the b-th sequence is valid and equal to 0 when this position corresponds to padding or another token that should be ignored. The mini-batch loss is

$$\widehat{\mathscr{L}}_{\mathrm{pre}}(\theta) = -\frac{\displaystyle\sum_{b=1}^{B}\sum_{t=1}^{T_{\max}} \mu_{b,t}\log \pi_{b,t,w_t^{(b)}}}{\displaystyle\sum_{b=1}^{B}\sum_{t=1}^{T_{\max}} \mu_{b,t}},$$

where $\pi_{b,t,w_t^{(b)}}$ is the probability assigned by the model to the observed target token $w_t^{(b)}$. Only positions with $\mu_{b,t} = 1$ contribute to the loss and its gradient.

The use of the true preceding tokens during training is called *teacher forcing*. At position t, the model is conditioned on

$$\big(\mathrm{BOS}, w_1, \ldots, w_{t-1}\big),$$

rather than on tokens that it generated itself. The causal mask nevertheless makes the prediction at each position independent of future tokens, which allows all next-token losses in a sequence to be evaluated simultaneously.

For computational efficiency, documents are often concatenated with EOS separators and divided into blocks of a fixed maximum length. This procedure, called *sequence packing*, reduces the amount of padding. When no additional attention mask is used, tokens in a packed block may attend to tokens belonging to an earlier document; the EOS token then serves as a learned boundary marker. If strict document independence is required, one may instead use a block-diagonal attention mask.

The objective above is self-supervised because the target token is obtained directly from the training text. It does not, however, distinguish truth from falsehood, useful behaviour from harmful behaviour, or a well-written answer from an appropriate answer to a user request. A model trained only with this objective is therefore a *base model*: it estimates regularities in its training distribution, but has not yet been specifically adapted to follow instructions or human preferences.

Remark 45. *Autoregressive next-token prediction differs from masked language modelling, in which selected tokens are predicted from both their left and right context. The latter objective is well suited to bidirectional encoder models, whereas the causal next-token objective directly defines a generative distribution over complete sequences.*

The next subsection studies how the pre-training loss evolves when the number of parameters, the number of training tokens, and the available computational budget are scaled.

7.3 Scaling laws, data, and compute

Pre-training a large language model requires three main resources P, D and C, where:

- P denotes the number of trainable model parameters,
- D the number of training tokens presented to the model,
- C the total training compute.

In this subsection, P refers primarily to non-embedding parameters, in accordance with the convention used in the original scaling-law experiments. When a different parameter-counting convention is used, it should always be stated explicitly.

A *scaling law* is an empirical relation describing how the validation loss changes as one or several of these resources are increased. It is not a theorem about all language models. Rather, it is a fitted approximation that is valid for a specified model family, optimizer, data mixture, tokenizer, and range of scales.

The first systematic study of scaling laws for autoregressive Transformer language models found that, when the other resources were not the limiting factor, the cross-entropy loss could be approximated by power laws of the form [9]:

$$L(P) \simeq \left(\frac{P_{\mathrm{c}}}{P}\right)^{\alpha_P}, \qquad L(D) \simeq \left(\frac{D_{\mathrm{c}}}{D}\right)^{\alpha_D}, \qquad L(C_{\min}) \simeq \left(\frac{C_{\mathrm{c}}^{\min}}{C_{\min}}\right)^{\alpha_C}.$$

Here, L denotes the validation cross-entropy loss (in nats per token), and the exponents $\alpha_P, \alpha_D, \alpha_C$ are positive. The quantity $C_{\min}$ is not an arbitrary compute budget: it denotes the approximately minimal compute required to reach a given loss when the remaining training choices are allocated efficiently.

The coefficients

$$P_{\mathrm{c}}, \qquad D_{\mathrm{c}}, \qquad C_{\mathrm{c}}^{\min}$$

and the fitted exponents are specific to the experimental setting. For instance, Kaplan *et al.* report $\alpha_P \approx 0.076$ with $P_{\mathrm{c}} \approx 8.8 \times 10^{13}$ non-embedding parameters, $\alpha_D \approx 0.095$ with $D_{\mathrm{c}} \approx 5.4 \times 10^{13}$ tokens, and $\alpha_C \approx 0.050$ with $C_{\mathrm{c}}^{\min} \approx 3.1 \times 10^{8}$ PF-days, where one PF-day (10^{15} floating-point operations per second during one day) represents about 8.64×10^{19} operations [9]. These values depend on the tokenizer, the data distribution, the model family, and the training procedure. For this reason, such numerical constants should not be interpreted as universal properties of language.

A useful joint approximation, employed in later compute-optimal scaling studies, is

$$L(P, D) \simeq L_\infty + aP^{-\alpha} + bD^{-\beta},$$

where

$$L_\infty \in \mathbb{R}, \qquad a, b, \alpha, \beta > 0.$$

The quantity L_∞ represents an irreducible loss floor for the chosen data distribution and model family. The term $aP^{-\alpha}$ describes the error caused by insufficient model capacity, whereas $bD^{-\beta}$ describes the error caused by insufficient training data.

For a dense decoder-only Transformer, the leading contribution to the training compute is often approximated by

$$C \simeq \kappa PD,$$

where κ is an implementation-dependent constant. Under common counting conventions, one often uses $\kappa \simeq 6$: approximately two floating-point operations per parameter are required in a forward pass, and the backward pass contributes a further factor of about three. This estimate neglects several effects, including optimizer operations, communication costs, and the context-length-dependent cost of attention. It should therefore be viewed as a useful first-order approximation, not as an exact accounting of hardware cost.

If a fixed compute budget C is available, then

$$D = \frac{C}{\kappa P}.$$

Substituting this relation into the joint scaling law gives a one-variable optimization problem:

$$\widetilde{L}_C(P) = L_\infty + aP^{-\alpha} + b\left(\frac{\kappa P}{C}\right)^{\beta}.$$

The first term depending on P decreases when the model becomes larger, whereas the second one increases: for fixed compute, a larger model leaves fewer tokens available for training.

Differentiating with respect to P yields

$$\frac{\mathrm{d}\widetilde{L}_C}{\mathrm{d}P} = -\alpha a P^{-\alpha-1} + \beta b \left(\frac{\kappa}{C}\right)^{\beta} P^{\beta-1}.$$

Hence, the compute-optimal parameter count is

$$P_\star(C) = \left(\frac{\alpha a}{\beta b}\right)^{\frac{1}{\alpha+\beta}} \left(\frac{C}{\kappa}\right)^{\frac{\beta}{\alpha+\beta}},$$

and the corresponding number of training tokens is

$$D_\star(C) = \left(\frac{\beta b}{\alpha a}\right)^{\frac{1}{\alpha+\beta}} \left(\frac{C}{\kappa}\right)^{\frac{\alpha}{\alpha+\beta}}.$$

These expressions formalize the central trade-off of LLM pre-training: increasing the number of parameters without increasing the training-token budget can produce an under-trained model, whereas increasing the amount of data without sufficient model capacity eventually yields diminishing returns.

The exact compute-optimal allocation has been revised as larger and more diverse training runs have become available. In particular, the Chinchilla study found that, within its experimental regime, the model size and the number of training tokens should grow at approximately the same rate: doubling the number of parameters should be accompanied by an approximate doubling of the number of training tokens [6]. In practice, this corresponds to roughly 20 training tokens per parameter: the Chinchilla model, with 70 billion parameters, was trained on 1.4 trillion tokens. This conclusion differs from earlier prescriptions that allocated a larger fraction of additional compute to model size.

The number of training tokens alone does not characterize the usefulness of a dataset. The data mixture introduced in the previous subsection matters substantially: duplication, low-quality text, domain mismatch, biased sources, contaminated evaluation examples, and the number of repeated passes over the same documents can all affect the observed loss. Thus, two runs with identical values of P, D, and C need not produce models of equal quality.

Remark 46. *Scaling laws are most directly concerned with a continuous quantity such as cross-entropy loss. They do not imply that every downstream benchmark score, reasoning behaviour, or safety property improves according to the same smooth curve. In particular, thresholded evaluation metrics can make a gradual improvement in likelihood appear as an abrupt change in capability.*

Scaling laws are nevertheless valuable because they allow small-scale experiments to inform the design of substantially larger training runs. The next stage in the development of an LLM is not merely to reduce its pre-training loss, but to adapt its behaviour to instructions and human preferences.

7.4 Post-training: instruction tuning and preference alignment

Pre-training produces a *base model* that approximates the distribution of its training corpus. Although such a model can often continue text coherently, it is not specifically trained to answer questions, follow a conversation, respect a desired style, or refuse an unsafe request. *Post-training* adapts this base model to a more targeted distribution of interactions and desired behaviours.

Let x denote a prompt, which may include a system message, a user request, and the preceding turns of a conversation. Let

$$y = (y_1, \dots, y_M)$$

denote a candidate response, including an end-of-sequence token. The conditional distribution induced by the language model is

$$\pi_\theta(y \mid x) = \prod_{t=1}^{M} p_\theta\big(y_t \mid x, y_{1:t-1}\big).$$

The notation π_θ emphasizes that, after post-training, the language model is viewed as a policy that maps a prompt to a distribution over possible responses.

Supervised fine-tuning. The first post-training stage is often *supervised fine-tuning* (SFT). It uses a dataset of demonstrations,

$$\mathscr{D}_{\mathrm{SFT}} = \left\{ \left(x^{(i)}, y^{(i)}\right) \right\}_{i=1}^{n},$$

where $x^{(i)}$ is an instruction or conversational context and $y^{(i)}$ is a response judged to be appropriate. The SFT objective is again a next-token prediction loss, but it is conditioned on the prompt:

$$\mathscr{L}_{\mathrm{SFT}}(\theta) = -\frac{1}{N_{\mathrm{resp}}} \sum_{i=1}^{n} \sum_{t=1}^{M_i} \log p_\theta\left(y_t^{(i)} \mid x^{(i)}, y_{1:t-1}^{(i)}\right),$$

where

$$N_{\mathrm{resp}} = \sum_{i=1}^{n} M_i$$

is the total number of response tokens in the dataset.

In a chat-formatted training example, the prompt and the response are usually concatenated into a single token sequence. However, the loss is commonly masked so that only the assistant response tokens contribute to $\mathscr{L}_{\mathrm{SFT}}$. This prevents the objective from allocating training signal to reproducing the user's prompt and focuses the update on the conditional distribution of desirable answers.

Supervised fine-tuning teaches the model to imitate demonstrations. It can improve instruction following, formatting, tone, and task-specific behaviour, but it does not by itself express trade-offs such as whether one of two plausible answers is more helpful, safer, or more precise.

Preference data and reward modelling. To represent such relative judgements, one can collect a preference dataset

$$\mathscr{D}_{\mathrm{pref}} = \left\{ \left(x^{(i)}, y_+^{(i)}, y_-^{(i)}\right) \right\}_{i=1}^{m},$$

where

$$y_+^{(i)} \succ_{x^{(i)}} y_-^{(i)}$$

means that evaluators prefer the response $y_+^{(i)}$ to the response $y_-^{(i)}$ for the prompt $x^{(i)}$.

In the classical reinforcement learning from human feedback (RLHF) pipeline, one first trains a scalar-valued reward model

$$r_\phi(x, y) \in \mathbb{R}.$$

A common assumption is the Bradley–Terry preference model,

$$\mathbb{P}_\phi\left(y_+ \succ_x y_-\right) = \sigma\left(r_\phi(x, y_+) - r_\phi(x, y_-)\right),$$

where

$$\sigma(u) = \frac{1}{1 + \exp(-u)}$$

is the logistic sigmoid function. The reward model is trained by minimizing the binary cross-entropy loss

$$\mathscr{L}_{\mathrm{RM}}(\phi) = -\frac{1}{m} \sum_{i=1}^{m} \log \sigma\left(r_\phi\left(x^{(i)}, y_+^{(i)}\right) - r_\phi\left(x^{(i)}, y_-^{(i)}\right)\right).$$

RLHF and KL regularization. Once a reward model has been learned, the policy can be optimized to favour responses with a high predicted reward. Let π_{ref} be a fixed reference policy, usually the SFT model, and let ρ denote the empirical distribution of prompts. A standard KL-regularized RLHF objective is

$$\mathscr{J}_{\mathrm{RLHF}}(\theta) = \mathbb{E}_{x \sim \rho}\left[\mathbb{E}_{y \sim \pi_\theta(\cdot|x)}\left[r_\phi(x, y)\right] - \beta D_{\mathrm{KL}}\left(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid x)\right)\right],$$

where $\beta > 0$ controls the strength of the regularization.

The KL term prevents the optimized policy from deviating too far from the reference model. Without such a constraint, the policy may exploit imperfections in the reward model and generate responses that receive a high predicted reward while being undesirable to a human evaluator. This phenomenon is often called *reward hacking*.

Note the analogy with the penalized problem (2) of Section 4.1: the KL term plays the role of a regularizer that keeps the policy close to π_{ref}.

For a fixed prompt x, the optimal policy associated with the KL-regularized objective has the form

$$\pi^\star(y \mid x) = \frac{\pi_{\mathrm{ref}}(y \mid x)\exp\big(r_\phi(x,y)/\beta\big)}{Z_\phi(x)},$$

where

$$Z_\phi(x) = \sum_{y'} \pi_{\mathrm{ref}}(y' \mid x)\exp\big(r_\phi(x,y')/\beta\big)$$

is a normalizing constant (see Exercise 5 of Section 7.7). Thus, the reward model exponentially reweights the responses that were already likely under the reference policy.

Direct preference optimization. Direct preference optimization (DPO) avoids fitting a separate reward model and avoids a separate reinforcement-learning stage. Instead, it directly optimizes the language model on preference pairs. Define

$$\Delta_\theta\big(x; y_+, y_-\big) = \log \frac{\pi_\theta(y_+ \mid x)}{\pi_{\mathrm{ref}}(y_+ \mid x)} - \log \frac{\pi_\theta(y_- \mid x)}{\pi_{\mathrm{ref}}(y_- \mid x)}.$$

The DPO loss is

$$\mathscr{L}_{\mathrm{DPO}}(\theta) = -\frac{1}{m}\sum_{i=1}^{m} \log \sigma\left(\beta\, \Delta_\theta\big(x^{(i)}; y_+^{(i)}, y_-^{(i)}\big)\right).$$

Minimizing this loss increases the relative probability of preferred responses and decreases that of rejected responses, compared with the reference policy. DPO can be derived from the same KL-regularized preference framework as RLHF (see Exercise 6 of Section 7.7), but it replaces explicit reward modelling and policy optimization with a single classification-like objective [18].

Instruction tuning and preference alignment are complementary. SFT provides explicit demonstrations of the desired response format and behaviour, whereas preference optimization learns relative judgements between alternative responses. The combination of supervised demonstrations, preference data, and KL regularization forms the basis of the post-training pipeline used in many instruction-following language models [16].

Remark 47. *Alignment does not guarantee factuality, harmlessness, or agreement among all users. Preference labels may be incomplete, noisy, culturally dependent, or inconsistent across contexts. Post-training changes the probability assigned to responses; it does not equip the model with a perfect external verifier of the claims it generates.*

After post-training has specified the response distribution $\pi_\theta(\cdot \mid x)$, a decoding procedure is required to select or sample an actual response. This is the subject of the next subsection.

7.5 Decoding, evaluation, and reliability

After pre-training and post-training, an LLM defines a conditional probability distribution

$$\pi_\theta(y \mid x) = \prod_{t=1}^{M} p_\theta\big(y_t \mid x, y_{1:t-1}\big)$$

over possible responses y to a prompt x. This distribution does not, by itself, specify a unique response. A *decoding strategy* is therefore required to select or sample tokens sequentially until an EOS token or another stopping condition is reached.

Decoding strategies. The simplest method is *greedy decoding*. At each position t, it selects the most likely next token,

$$\widehat{y}_t = \operatorname*{argmax}_{v \in \mathscr{V}} p_\theta\big(v \mid x, \widehat{y}_{1:t-1}\big).$$

Greedy decoding is computationally inexpensive and deterministic, but it does not generally maximize the probability of the complete sequence,

$$\widehat{y}_{\mathrm{MAP}} \in \operatorname*{argmax}_{y \in \mathscr{V}^*} \log \pi_\theta(y \mid x).$$

Indeed, a locally optimal token choice can prevent a more probable continuation from being reached later in the sequence (see Exercise 8 of Section 7.7).

A common alternative is to modify the sharpness of the next-token distribution through a temperature parameter $\tau > 0$. Given the logits $\ell_t \in \mathbb{R}^{|\mathscr{V}|}$, define

$$\pi_t^{(\tau)}(v) = \frac{\exp\bigl(\ell_{t,v}/\tau\bigr)}{\displaystyle\sum_{v' \in \mathscr{V}} \exp\bigl(\ell_{t,v'}/\tau\bigr)}.$$

For small values of τ, the distribution becomes increasingly concentrated on the most likely tokens. For large values of τ, the distribution becomes flatter and sampling produces more diverse, but potentially less reliable, continuations.

Sampling from the full vocabulary may assign non-negligible probability to a very large number of implausible tokens. Two common truncation methods are therefore *top-k* sampling and *nucleus sampling*. Let

$$K_t \subseteq \mathscr{V}$$

be the set of the k most likely tokens under π_t. The top-k distribution is

$$\pi_t^{(k)}(v) = \frac{\pi_t(v)\,\mathbf{1}_{\{v \in K_t\}}}{\displaystyle\sum_{u \in K_t} \pi_t(u)}.$$

The value of k is fixed at every generation step.

For nucleus sampling, also called top-p sampling, let

$$\pi_t\bigl(v_{(1)}\bigr) \geqslant \pi_t\bigl(v_{(2)}\bigr) \geqslant \cdots$$

be the tokens ordered by decreasing probability. Given a threshold $p_{\mathrm{nuc}} \in (0, 1]$, define

$$r_t = \min\left\{ r \in \{1, \ldots, |\mathscr{V}|\} : \sum_{j=1}^{r} \pi_t\bigl(v_{(j)}\bigr) \geqslant p_{\mathrm{nuc}} \right\}.$$

The nucleus is then

$$A_t(p_{\mathrm{nuc}}) = \bigl\{ v_{(1)}, \ldots, v_{(r_t)} \bigr\},$$

and the distribution is renormalized over this set. Unlike top-k sampling, the number of available tokens adapts to the uncertainty of the model at each position [7].

Beam search is another widely used strategy. Rather than retaining a single partial continuation, it maintains the B most likely partial sequences at each step. Beam search provides an approximation to sequence-level maximum-likelihood decoding, but high likelihood is not always synonymous with a useful or natural response. In open-ended generation, an excessively likelihood-oriented decoder can favour short, generic, or repetitive outputs.

Evaluation. An LLM should be evaluated as a complete system, not only as a probability distribution. Let

$$\mathscr{E} = \Bigl\{ \bigl(x^{(i)}, y^{\star(i)}\bigr) \Bigr\}_{i=1}^{n}$$

be an evaluation set and let d denote a fixed decoding procedure. The generated response is

$$\widehat{y}^{(i)} = d\bigl(\pi_\theta(\cdot \mid x^{(i)})\bigr).$$

For a task-dependent scoring function

$$s : \mathscr{V}^* \times \mathscr{V}^* \longrightarrow \mathbb{R},$$

the empirical score is

$$\widehat{S} = \frac{1}{n} \sum_{i=1}^{n} s\big(\widehat{y}^{(i)}, y^{\star(i)}\big).$$

For example, choosing

$$s(\widehat{y}, y^{\star}) = \mathbf{1}_{\{\widehat{y}=y^{\star}\}}$$

gives exact-match accuracy. Other tasks may instead require token-level F_1 scores, ranking metrics, code execution, mathematical verification, or a human evaluation rubric.

The validation cross-entropy loss and perplexity introduced earlier are *intrinsic* metrics: they measure how well the model predicts text from a specified distribution. Task scores are *extrinsic* metrics: they measure how well a fully specified system performs on a particular objective. A lower perplexity does not automatically imply better instruction following, factuality, or safety.

A meaningful evaluation protocol should specify at least the prompt template, the decoding parameters, the number of samples, the post-processing rules, and any available retrieval or tool-use mechanism. It should also guard against data contamination: an apparent evaluation gain is not informative if the evaluation examples, or near-duplicates of them, appeared in the pre-training or post-training data.

Calibration and selective prediction. A reliable system should not only be accurate on average; it should also indicate when its answers are likely to be unreliable. Let

$$z_i = \mathbf{1}_{\{\widehat{y}^{(i)} \text{ is correct}\}}$$

and let $c_i \in [0,1]$ be a confidence score assigned to the response. Perfect calibration would mean that

$$\mathbb{P}\big(z_i = 1 \mid c_i = q\big) = q$$

for every confidence value q.

In practice, confidence values are grouped into bins

$$B_1, \ldots, B_R.$$

For a bin B_r, define

$$\mathrm{acc}(B_r) = \frac{1}{|B_r|} \sum_{i \in B_r} z_i, \qquad \mathrm{conf}(B_r) = \frac{1}{|B_r|} \sum_{i \in B_r} c_i.$$

The expected calibration error is then

$$\mathrm{ECE} = \sum_{r=1}^{R} \frac{|B_r|}{n} \left|\mathrm{acc}(B_r) - \mathrm{conf}(B_r)\right|.$$

A low value of ECE indicates that confidence is better aligned with empirical correctness, although it depends on the choice of bins and on the evaluation distribution [4].

For free-form language generation, confidence is more difficult to define than in ordinary classification. The probability assigned to a generated sequence is affected by its length, tokenization, and decoding procedure; it is not, by itself, the probability that the factual claims in the response are true. Useful confidence estimates may instead rely on a task-specific verifier, agreement across multiple samples, an external tool, or a calibrated uncertainty model.

If a system is allowed to abstain, a confidence threshold γ can be used to define the selective risk

$$\mathrm{Risk}(\gamma) = \frac{\displaystyle\sum_{i=1}^{n} \mathbf{1}_{\{c_i \geqslant \gamma\}} (1 - z_i)}{\displaystyle\sum_{i=1}^{n} \mathbf{1}_{\{c_i \geqslant \gamma\}}}.$$

A reliable selective system should reduce its error rate as the confidence threshold increases, while preserving useful coverage of the input distribution.

Grounding and external knowledge. A language model stores information only implicitly in its parameters, and this information may be incomplete, outdated, or incorrectly recalled. Retrieval-augmented generation introduces an external evidence variable z, such as a set of retrieved documents, and models

$$p_{\theta,\eta}(y \mid x) = \sum_{z \in \mathscr{Z}} p_\eta(z \mid x)\, p_\theta(y \mid x, z),$$

where $p_\eta(z \mid x)$ is the retriever distribution and $p_\theta(y \mid x, z)$ is the conditional language model. In practical systems, a small number of highly ranked documents is retrieved and inserted into the model context [13].

Retrieval, citations, calculators, databases, and code-execution tools can improve reliability when they provide relevant and verifiable evidence. However, they do not offer an automatic guarantee of correctness: a retriever may return irrelevant or outdated material, and a generator may misinterpret, ignore, or inaccurately summarize the retrieved evidence.

Remark 48. *The temperature used for generation should not be confused with the post-hoc temperature scaling sometimes used for confidence calibration. The former changes the distribution from which tokens are sampled; the latter is a calibration procedure applied to predicted scores.*

Decoding, evaluation, and reliability are therefore inseparable. A model may have a low next-token prediction loss while producing poor answers under an inappropriate decoding rule, while an apparently capable system may remain unreliable if its evaluation protocol fails to test the relevant forms of error. These considerations also remain essential when language models are extended to process images, audio, or other modalities.

7.6 Capabilities and limitations

The Transformer architecture quickly became the standard backbone of language models. Successive generations of decoder-only models, such as GPT-1, GPT-2 and GPT-3, showed that jointly scaling the number of parameters, the amount of data and the compute, in the regime described in Section 7.3, steadily improves performance on downstream tasks. Figure 15 gives a non-exhaustive overview of the resulting capabilities.

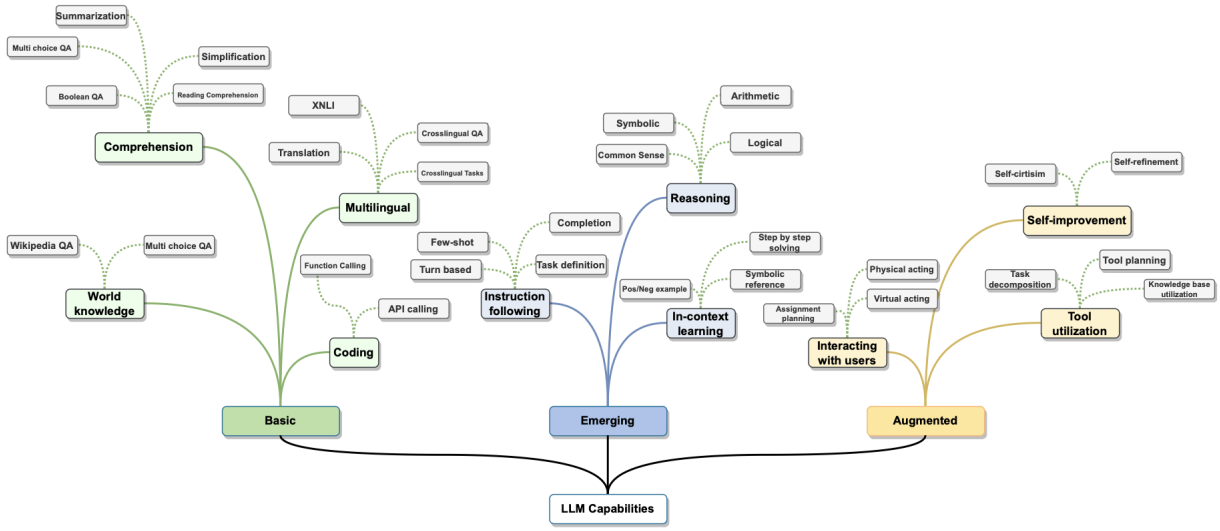


Figure 15: Overview of LLM capabilities.

Some abilities of large models are absent, or barely measurable, in small models. Three of them are often highlighted [27]:

- **In-context learning.** This ability was popularized by GPT-3 [1]. Given an instruction and, possibly, a few input–output demonstrations in its prompt, the model produces the expected output for a new input simply by continuing the text, without any additional training or gradient update.
- **Instruction following.** After fine-tuning on a mixture of tasks described in natural language (*instruction tuning*, see Section 7.4), a model can perform new tasks that are described only by an instruction, without examples.

- **Step-by-step reasoning.** Prompting a model to produce intermediate reasoning steps before its final answer, a technique called *chain-of-thought* prompting [25], markedly improves the performance of large models on multi-step problems such as mathematical word problems, whereas it brings little benefit to small models.

These abilities are often called *emergent*. This term should be used with care. As noted in Remark 46, an abrupt change in a thresholded metric such as exact-match accuracy may result from a smooth improvement of the underlying likelihood, and several apparently emergent abilities become predictable when they are measured with continuous metrics [21].

LLMs also suffer from important limitations:

- **Hallucinations.** LLMs may generate plausible-looking statements that contradict the available sources or cannot be verified. This issue can be partially mitigated by alignment and by grounding the model with retrieval or external tools (Section 7.5).
- **Knowledge recency.** The parametric knowledge of an LLM is frozen at training time. Retrieving external, up-to-date sources is a practical workaround, but reliably updating the knowledge stored in the parameters remains an open research problem.
- **Reasoning inconsistency.** An LLM may reach a correct answer through an invalid reasoning path, or a wrong answer after a correct one. Mitigations include training with feedback on the reasoning steps, sampling several reasoning paths and aggregating their answers, and verifying intermediate results with external tools.
- **Toxic and biased content.** Since pre-training corpora mix high-quality and low-quality data, LLMs can reproduce toxic, biased, or harmful content.

These limitations motivate the post-training methods of Section 7.4 and the reliability considerations of Section 7.5. A common summary of the goal of alignment is that an assistant should be *helpful*, *honest* and *harmless*.

7.7 Exercises

1. **Cross-entropy and Kullback–Leibler divergence.** Let q and p be two probability distributions on a finite set, with $p > 0$. Show that
$$H(q, p) = H(q) + D_{\mathrm{KL}}(q\|p),$$
and that $D_{\mathrm{KL}}(q\|p) \geqslant 0$, with equality if and only if $p = q$ (Hint: use Jensen's inequality). Deduce the minimizer of $\mathscr{R}_{\mathrm{pre}}$ over all probability distributions.

2. **Perplexity.** Show that a model which predicts the uniform distribution over $\mathscr{V}$ at every position has perplexity $|\mathscr{V}|$. Interpret the perplexity as an effective number of equally likely candidates for the next token. Why can perplexities computed with two different tokenizers not be compared directly?

3. **Compute-optimal allocation.**
   (a) Show that $\widetilde{L}_C$ has a unique minimizer on $(0, +\infty)$ and recover the expressions of $P_\star(C)$ and $D_\star(C)$.
   (b) Show that
$$\frac{D_\star(C)}{P_\star(C)} = \left(\frac{\beta b}{\alpha a}\right)^{\frac{2}{\alpha+\beta}} \left(\frac{C}{\kappa}\right)^{\frac{\alpha-\beta}{\alpha+\beta}}.$$
   (c) Deduce that the optimal number of training tokens per parameter does not depend on the compute budget if and only if $\alpha = \beta$. Relate this result to the conclusion of the Chinchilla study.

4. **Order of magnitude of the training compute.** Using $C \simeq 6PD$, estimate the training compute of a model with 70 billion parameters trained on 1.4 trillion tokens. Express the result in floating-point operations and in PF-days.

5. **Optimal KL-regularized policy.** Fix a prompt x and assume that the set $\mathscr{Y}$ of possible responses is finite. For a distribution π on $\mathscr{Y}$, let
$$J(\pi) = \mathbb{E}_{y\sim\pi}\big[r_\phi(x, y)\big] - \beta\, D_{\mathrm{KL}}\big(\pi \,\|\, \pi_{\mathrm{ref}}(\cdot \mid x)\big).$$
Show that $J(\pi) = \beta \log Z_\phi(x) - \beta\, D_{\mathrm{KL}}\big(\pi \,\|\, \pi^\star(\cdot \mid x)\big)$. Deduce that $\pi^\star(\cdot \mid x)$ is the unique maximizer of J.

6. **From RLHF to DPO.**

(a) Using the expression of $\pi^\star$, express $r_\phi(x, y)$ as a function of $\pi^\star(y \mid x)$, $\pi_{\text{ref}}(y \mid x)$ and $Z_\phi(x)$.

(b) Substitute this expression into the Bradley–Terry model and show that the normalizing constant $Z_\phi(x)$ cancels.

(c) Deduce that, if the policy π_θ is used in place of $\pi^\star$, minimizing the negative log-likelihood of the preference data leads to the loss $\mathscr{L}_{\text{DPO}}$.

7. **Temperature.** Let $\ell \in \mathbb{R}^{|\mathscr{V}|}$ be a vector of logits. Show that, when $\tau \to 0^+$, the distribution $\pi^{(\tau)}$ converges to the uniform distribution on $\operatorname{argmax}_v \ell_v$, and that, when $\tau \to +\infty$, it converges to the uniform distribution on $\mathscr{V}$.

8. **Greedy decoding is not MAP decoding.** Construct an example with a vocabulary of two tokens and responses of length two for which greedy decoding does not return the most probable response.

8 Multimodal extensions of LLMs

The LLMs studied so far model and generate sequences of text tokens. A *multimodal large language model* extends this principle by converting non-text inputs, such as images, audio signals, videos, or sensor measurements, into continuous representations that can condition a language model. The LLM therefore remains responsible for modelling and generating text, while a modality-specific encoder extracts information from the additional input.

Let o denote an observation from an arbitrary modality and let

$$E_{\text{mod}}(o)$$

be an encoder producing a sequence of features. A bridge or projection module Π_ψ maps these features to the hidden dimension of the LLM:

$$Z_{\text{mod}} = \Pi_\psi\big(E_{\text{mod}}(o)\big) \in \mathbb{R}^{d_{\text{LLM}} \times M}.$$

The columns of Z_{mod} are often called *modality tokens*. They are continuous vectors, not discrete elements of the text vocabulary.

If

$$X_{\text{text}}(x) \in \mathbb{R}^{d_{\text{LLM}} \times T}$$

denotes the embedding matrix associated with a text prompt x, a common strategy is to concatenate the modality and text representations:

$$\widetilde{X} = \big[Z_{\text{mod}} \quad X_{\text{text}}(x)\big] \in \mathbb{R}^{d_{\text{LLM}} \times (M+T)}.$$

The language model can then generate a response $y_{1:L}$ according to

$$p_\theta\big(y_{1:L} \mid o, x\big) = \prod_{t=1}^{L} p_\theta\big(y_t \mid \widetilde{X}, y_{1:t-1}\big).$$

This formulation makes clear that multimodality changes the conditioning context of the language model, rather than replacing its autoregressive generation mechanism.

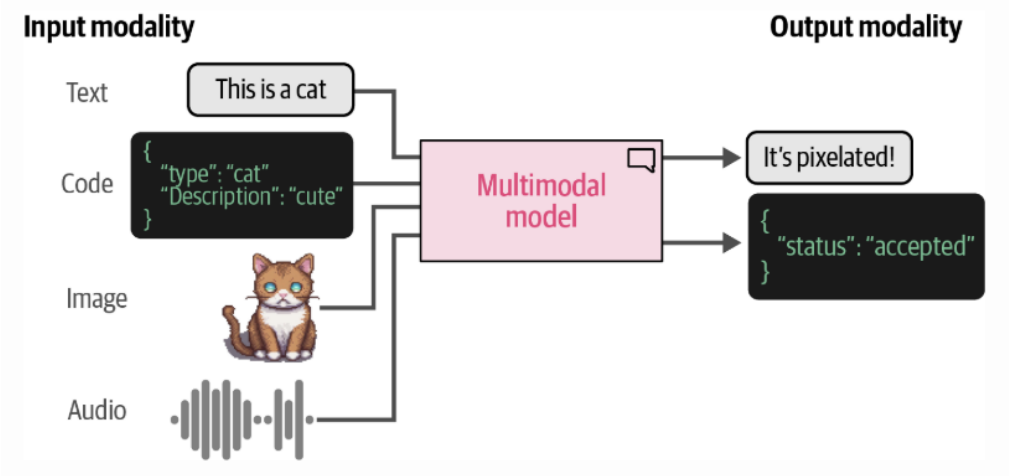


Figure 16: A multimodal LLM combines modality-specific encoders with a language model. The encoded observations are transformed into continuous tokens that condition text generation.

8.1 From images to visual tokens

We now focus on images. Let

$$I \in \mathbb{R}^{H \times W \times C}$$

be an image of height H, width W, and C channels. Suppose that H and W are divisible by a patch size P. The image can then be partitioned into

$$N_{\text{patch}} = \frac{H}{P}\frac{W}{P}$$

non-overlapping patches. After flattening, the i-th patch is represented by

$$p_i \in \mathbb{R}^{P^2C}.$$

Using the column-token convention, the patch matrix is

$$\mathsf{P} = \begin{bmatrix} p_1 & p_2 & \cdots & p_{N_{\text{patch}}} \end{bmatrix} \in \mathbb{R}^{P^2C \times N_{\text{patch}}}.$$

A learned linear map embeds every patch into a vector of dimension d_{vis}:

$$W_{\text{patch}} \in \mathbb{R}^{d_{\text{vis}} \times P^2C}, \qquad b_{\text{patch}} \in \mathbb{R}^{d_{\text{vis}}}.$$

If

$$R = \begin{bmatrix} r_1 & r_2 & \cdots & r_{N_{\text{patch}}} \end{bmatrix} \in \mathbb{R}^{d_{\text{vis}} \times N_{\text{patch}}}$$

denotes a matrix of positional embeddings, the input to a Vision Transformer (ViT) is

$$X_{\text{vis}}^{(0)} = W_{\text{patch}}\mathsf{P} + b_{\text{patch}}\mathbf{1}_{N_{\text{patch}}}^{\top} + R \in \mathbb{R}^{d_{\text{vis}} \times N_{\text{patch}}},$$

where $\mathbf{1}_{N_{\text{patch}}}$ is the vector of ones in $\mathbb{R}^{N_{\text{patch}}}$. A Transformer encoder then maps this sequence to contextual visual features,

$$F = E_{\text{vis}}\big(X_{\text{vis}}^{(0)}\big) \in \mathbb{R}^{d_{\text{vis}} \times N_{\text{patch}}}.$$

Remark 49. *A visual patch is not analogous to a text token in every respect. Text tokens are discrete symbols selected from a shared vocabulary and embedded through a lookup table. Image patches are continuous arrays of pixel values, so the same linear patch-embedding map is applied to every patch. Their representations become meaningful through the visual encoder and the image data on which it is trained. Note also that applying the same linear map to every non-overlapping patch is exactly a convolution with kernel size and stride equal to P (Section 5).*

Vision Transformers treat an image as a sequence of patches and can therefore reuse the Transformer machinery introduced earlier [2]. To connect a ViT to an LLM, the visual features F must still be transformed into the hidden dimension expected by the language model. This transformation may be a simple linear projection, a multilayer perceptron, a cross-attention module, or a dedicated bridge such as the Q-Former introduced below.

We next study two complementary vision-language models. CLIP is a *contrastive dual-encoder* model that aligns images and text in a shared embedding space. BLIP-2 is a *generative vision-language model* that uses visual features as additional context for text generation.

8.2 Contrastive vision-language models: CLIP

CLIP consists of an image encoder and a text encoder. Given a batch of B paired examples

$$\big\{(I_i, s_i)\big\}_{i=1}^{B},$$

the image encoder and text encoder produce vectors

$$\bar{v}_i = f_\phi(I_i), \qquad \bar{u}_i = g_\psi(s_i).$$

After projection and normalization, we obtain

$$v_i = \frac{\bar{v}_i}{\|\bar{v}_i\|_2}, \qquad u_i = \frac{\bar{u}_i}{\|\bar{u}_i\|_2},$$

where $v_i, u_i \in \mathbb{R}^d$. Thus,

$$v_i^{\top} u_j$$

is the cosine similarity between the image I_i and the text s_j.

Using the column convention, define

$$V = \begin{bmatrix} v_1 & v_2 & \cdots & v_B \end{bmatrix}, \qquad U = \begin{bmatrix} u_1 & u_2 & \cdots & u_B \end{bmatrix} \in \mathbb{R}^{d \times B},$$

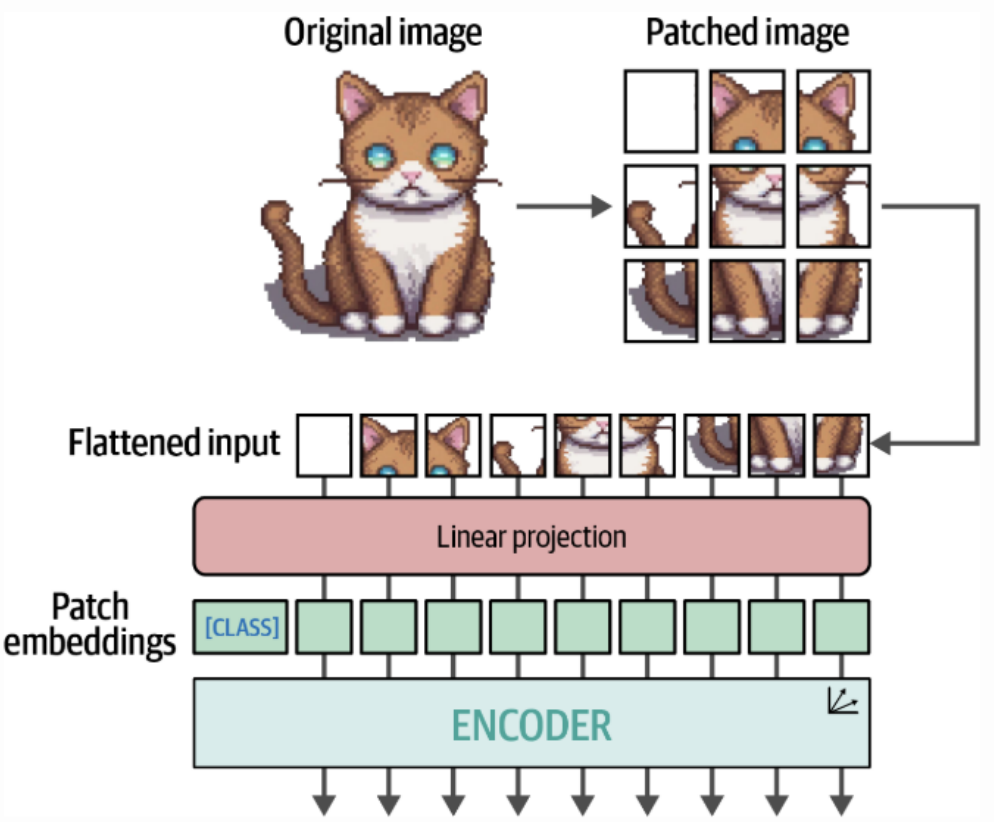


Figure 17: An image is split into patches, embedded as vectors, and processed by a vision encoder. A projection or bridge module then transforms the visual features into tokens that can condition an LLM.

and let

$$S = \frac{V^\top U}{\tau} \in \mathbb{R}^{B \times B},$$

where $\tau > 0$ is a temperature parameter. The entry

$$S_{ij} = \frac{v_i^\top u_j}{\tau}$$

measures the similarity between the i-th image and the j-th text in the batch.

The CLIP objective is the symmetric cross-entropy loss

$$\mathscr{L}_{\text{CLIP}} = -\frac{1}{2B} \sum_{i=1}^{B} \left[\log \left(\frac{\exp(S_{ii})}{\sum_{j=1}^{B} \exp(S_{ij})} \right) + \log \left(\frac{\exp(S_{ii})}{\sum_{j=1}^{B} \exp(S_{ji})} \right) \right].$$

The first term treats each image as a query and asks the model to identify its paired text among the B texts in the batch. The second term treats each text as a query and asks the model to identify its paired image. Hence, matched image-text pairs are encouraged to have high similarity, whereas mismatched pairs act as in-batch negatives.

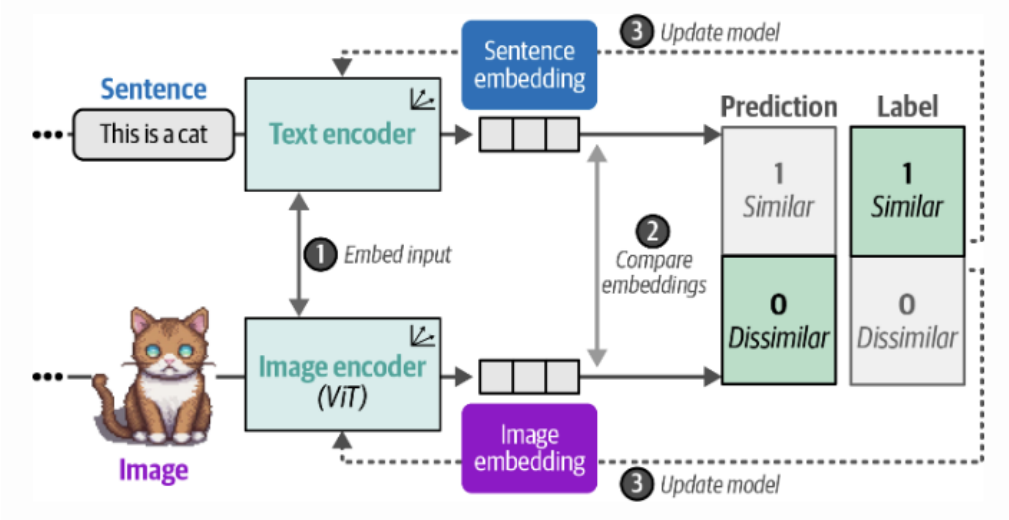


Figure 18: Contrastive training of CLIP. The image and text encoders are trained jointly so that matched image-text pairs have a higher similarity than mismatched pairs in the same batch.

CLIP can be used for image retrieval, text retrieval, semantic search, and zero-shot classification. For the latter, let $\mathscr{C}$ be a set of candidate classes and let u_c be the embedding of a textual description of class c. Given an image embedding v, the predicted class is

$$\hat{c} = \operatorname*{argmax}_{c \in \mathscr{C}} v^\top u_c.$$

Natural language therefore provides the classifier weights at inference time.

CLIP is not itself an image-generation model. It learns aligned image and text representations, which can be used by a separate generative system as a guidance signal. The original CLIP model was trained on 400 million image-text pairs and demonstrated strong zero-shot transfer to a broad range of visual classification tasks [17].

8.3 Generative vision-language models: BLIP-2

Unlike CLIP, BLIP-2 is designed to generate text conditioned on an image and, optionally, a textual prompt. Its central idea is to connect a frozen pre-trained vision encoder and a frozen LLM through a lightweight *Querying Transformer*, or *Q-Former* [14].

Let

$$F = E_{\text{vis}}(I) \in \mathbb{R}^{d_{\text{vis}} \times N_{\text{patch}}}$$

be the output of the frozen vision encoder. The Q-Former contains M learnable query vectors,

$$Q^{(0)} = \begin{bmatrix} q_1 & q_2 & \cdots & q_M \end{bmatrix} \in \mathbb{R}^{d_q \times M}.$$

These queries interact with one another through self-attention and attend to the frozen visual features through cross-attention. The resulting query representations are

$$Z = \mathrm{QFormer}_{\psi}\big(F, Q^{(0)}\big) \in \mathbb{R}^{d_q \times M}.$$

The Q-Former contains an image Transformer and a text Transformer that share their self-attention layers. The image Transformer contains cross-attention layers through which the learnable queries access the visual features. The text Transformer can act as a text encoder or a text decoder, depending on the pre-training objective.

A projection layer maps the output of the Q-Former to the hidden dimension of the LLM:

$$Z_{\mathrm{vis}} = W_{\mathrm{proj}} Z + b_{\mathrm{proj}} \mathbf{1}_M^{\top} \in \mathbb{R}^{d_{\mathrm{LLM}} \times M},$$

where

$$W_{\mathrm{proj}} \in \mathbb{R}^{d_{\mathrm{LLM}} \times d_q}, \qquad b_{\mathrm{proj}} \in \mathbb{R}^{d_{\mathrm{LLM}}}.$$

The projected vectors are prepended to the text embeddings and act as *soft visual prompts* for the frozen LLM.

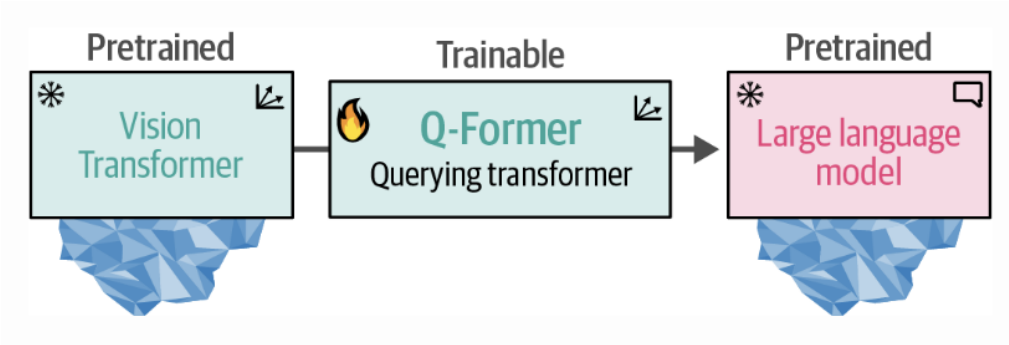


Figure 19: BLIP-2 uses a Q-Former and a projection layer as a trainable bridge between a frozen vision encoder and a frozen LLM.

BLIP-2 pre-trains this bridge in two stages.

Stage 1: vision-language representation learning. In the first stage, the vision encoder remains frozen and the Q-Former is trained on image-text pairs. The objective combines image-text contrastive learning, image-text matching, and image-grounded text generation:

$$\mathscr{L}_{\mathrm{stage\,1}} = \lambda_{\mathrm{ITC}} \mathscr{L}_{\mathrm{ITC}} + \lambda_{\mathrm{ITM}} \mathscr{L}_{\mathrm{ITM}} + \lambda_{\mathrm{ITG}} \mathscr{L}_{\mathrm{ITG}},$$

where the coefficients λ_{ITC}, λ_{ITM}, and λ_{ITG} balance the three losses.

For image-text contrastive learning, each image has several query outputs. If $z_{i,m}$ is the m-th query output of the i-th image and u_j is the text representation of the j-th caption, BLIP-2 uses a query-to-text similarity of the form

$$s_{ij}^{\mathrm{ITC}} = \frac{1}{\tau} \max_{1 \le m \le M} \frac{z_{i,m}^{\top} u_j}{\|z_{i,m}\|_2 \|u_j\|_2}.$$

A contrastive loss analogous to $\mathscr{L}_{\mathrm{CLIP}}$ is then applied to these similarities.

Image-text matching is a binary classification problem: the model must decide whether an image-caption pair is matched or mismatched. It encourages fine-grained interactions between the visual queries and the text representation. Image-grounded text generation uses a causal language-modelling loss to predict text from the visual queries:

$$\mathscr{L}_{\mathrm{ITG}} = -\sum_{t=1}^{L} \log p_{\psi}\big(s_t \mid Z, s_{1:t-1}\big).$$

Remark 50. *The frozen vision encoder is not modified during this stage. Rather than injecting textual information into the ViT itself, the Q-Former learns to extract from the fixed visual features the information that is most useful for representing and generating text.*

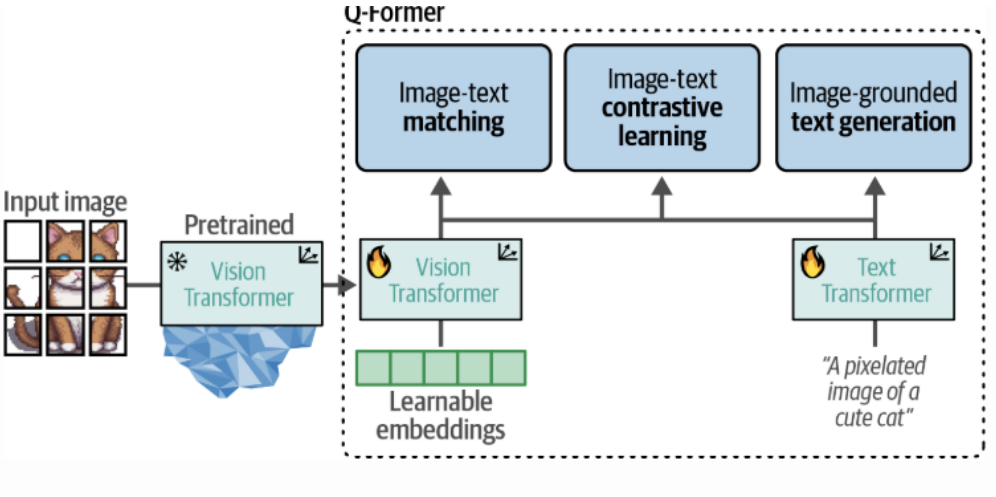


Figure 20: First stage of BLIP-2 training. A frozen vision encoder provides image features, while the Q-Former is trained through contrastive, matching, and image-grounded text-generation objectives.

Stage 2: vision-to-language generation. In the second stage, the projected visual tokens Z_{vis} are connected to a frozen LLM. The Q-Former and projection layer are trained so that their outputs can be interpreted by the language model. For a target text sequence $y_{1:L}$, the objective is

$$\mathscr{L}_{\text{stage 2}} = -\sum_{t=1}^{L} \log p_\theta\big(y_t \mid Z_{\text{vis}}, x, y_{1:t-1}\big),$$

where θ denotes the fixed parameters of the LLM. The trainable bridge therefore learns to translate visual information into a sequence of soft prompts that conditions the LLM's existing text-generation capability.

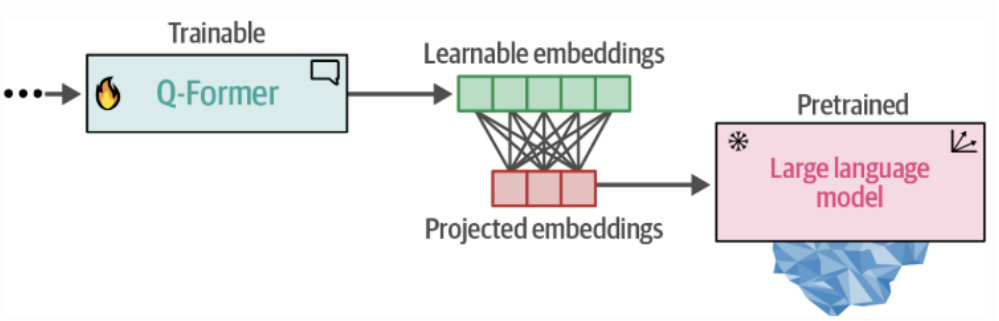


Figure 21: Second stage of BLIP-2 training. The Q-Former outputs are projected to the LLM hidden dimension and used as soft visual prompts for a frozen language model.

BLIP-2 illustrates a general design pattern for multimodal LLMs: retain strong unimodal encoders, learn a relatively small interface between them, and train the interface to produce representations that the LLM can use during generation. Other vision-language systems differ mainly in the architecture of this interface, for example by using a simple projector, a learned resampler, or cross-attention layers inserted into the language model.

8.4 Exercises

1. **Patch embedding as a convolution.** Show that the map $I \mapsto W_{\text{patch}}\mathsf{P} + b_{\text{patch}}\mathbf{1}_{N_{\text{patch}}}^{\top}$ can be computed by a convolutional layer (Section 5). Give its kernel size, stride, and numbers of input and output channels.

2. **The CLIP loss.**

   (a) Show that, if all the similarities S_{ij} are equal, then $\mathscr{L}_{\text{CLIP}} = \log B$.

(b) Using the fact that cosine similarities lie in $[-1, 1]$, show that

$$\mathscr{L}_{\text{CLIP}} \geqslant \log\bigl(1 + (B-1)\, e^{-2/\tau}\bigr).$$

Why must the temperature τ be small, and why is it usually learned during training?

(c) Show that CLIP zero-shot classification is a linear classifier on the normalized image embedding v, and identify its weights.

3. **Number of visual tokens.** An image of size 224×224 is processed by a ViT with patch size $P = 14$, and a Q-Former with $M = 32$ queries. Compute N_{patch}. Compare the cost of the self-attention layers of the LLM, for a text prompt of T tokens, when the visual features are prepended directly (after a linear projection) or through the Q-Former.

References

[1] T. B. Brown, B. Mann, N. Ryder, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, 2020.

[2] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al. An image is worth 16×16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.

[3] J. Duchi, E. Hazan, and Y. Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12:2121–2159, 2011.

[4] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning*, 2017.

[5] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2016.

[6] J. Hoffmann, S. Borgeaud, A. Mensch, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, 2022.

[7] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations*, 2020.

[8] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning*, 2015.

[9] J. Kaplan, S. McCandlish, T. Henighan, et al. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[10] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

[11] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, 2012.

[12] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[13] P. Lewis, E. Perez, A. Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020.

[14] J. Li, D. Li, S. Savarese, and S. Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International Conference on Machine Learning*, 2023.

[15] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

[16] L. Ouyang, J. Wu, X. Jiang, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022.

[17] A. Radford, J. W. Kim, C. Hallacy, et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, 2021.

[18] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, 2023.

[19] S. J. Reddi, S. Kale, and S. Kumar. On the convergence of Adam and beyond. In *International Conference on Learning Representations*, 2018.

[20] S. Santurkar, D. Tsipras, A. Ilyas, and A. Madry. How does batch normalization help optimization? In *Advances in Neural Information Processing Systems*, 2018.

[21] R. Schaeffer, B. Miranda, and S. Koyejo. Are emergent abilities of large language models a mirage? In *Advances in Neural Information Processing Systems*, 2023.

[22] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations*, 2015.

[23] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929–1958, 2014.

[24] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[25] J. Wei, X. Wang, D. Schuurmans, et al. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.

[26] A. C. Wilson, R. Roelofs, M. Stern, N. Srebro, and B. Recht. The marginal value of adaptive gradient methods in machine learning. In *Advances in Neural Information Processing Systems*, 2017.

[27] W. X. Zhao, K. Zhou, J. Li, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.